AI VIET NAM
@aivietnam.edu.vn

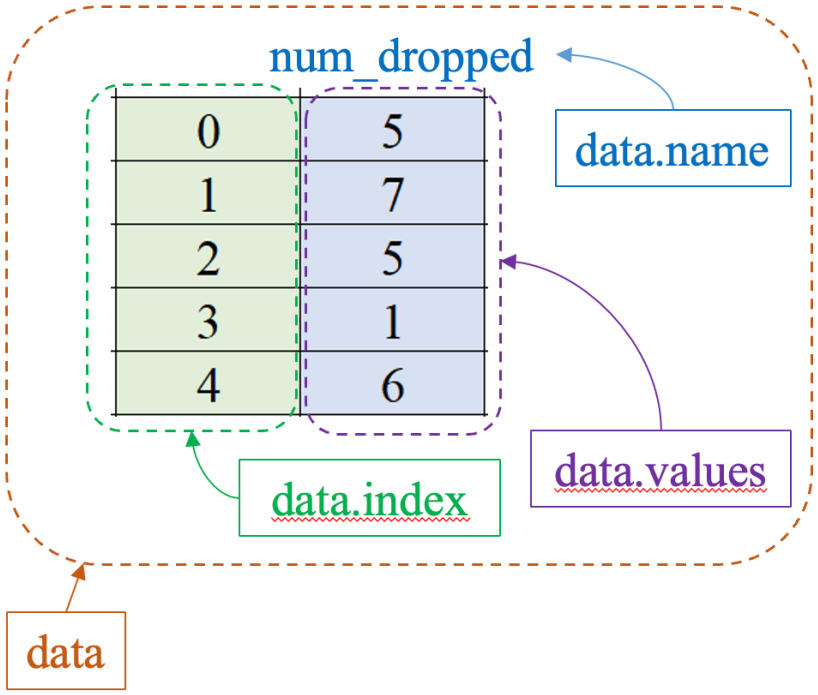
Data Visualization and Analysis Using Pandas

Quang-Vinh Dinh
Ph.D. in Computer Science

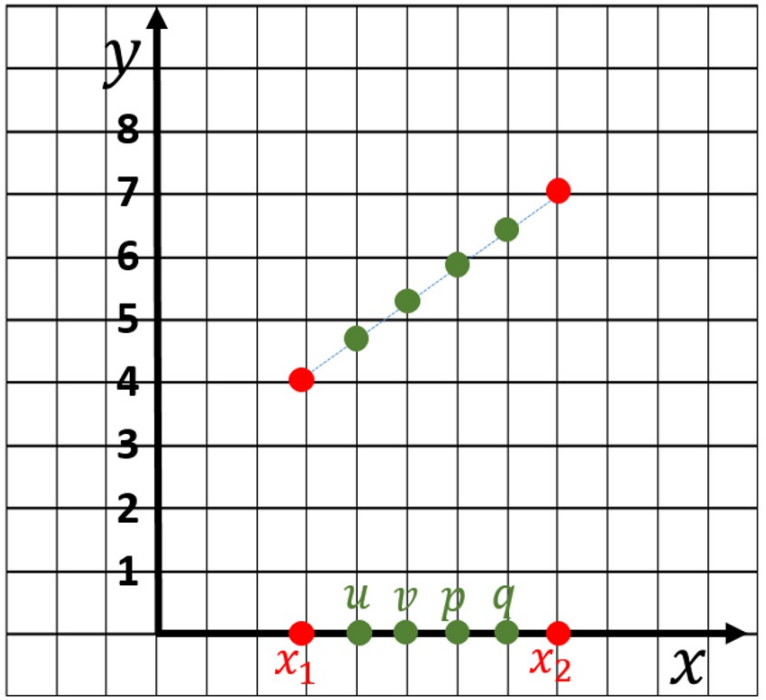
Year 2024

Objectives

Series in Pandas



Operations



Nội suy theo hàm tuyến tính

Case Studies

Outline

SECTION 1

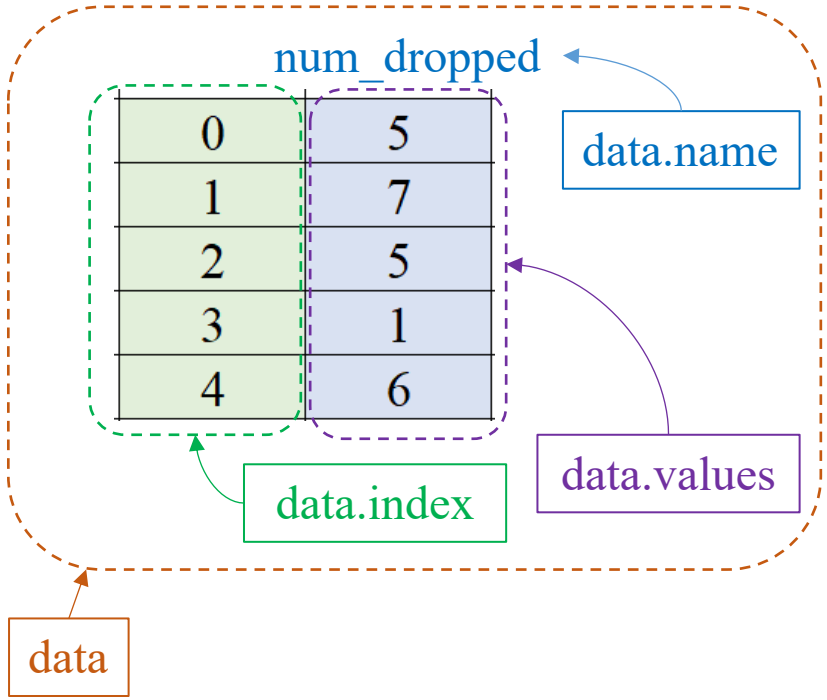
Series in Pandas

SECTION 2

Operations

SECTION 3

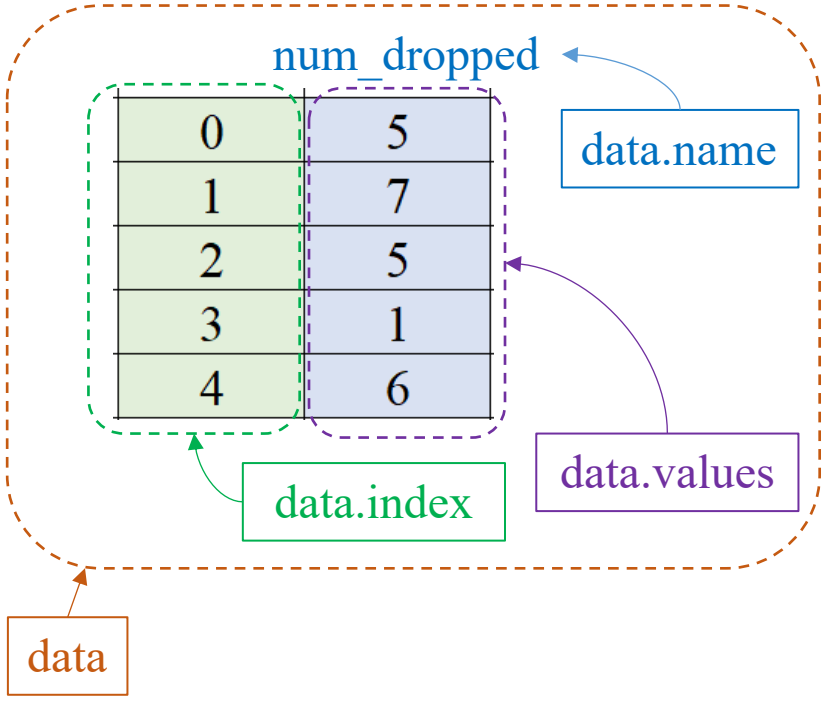
Case Studies



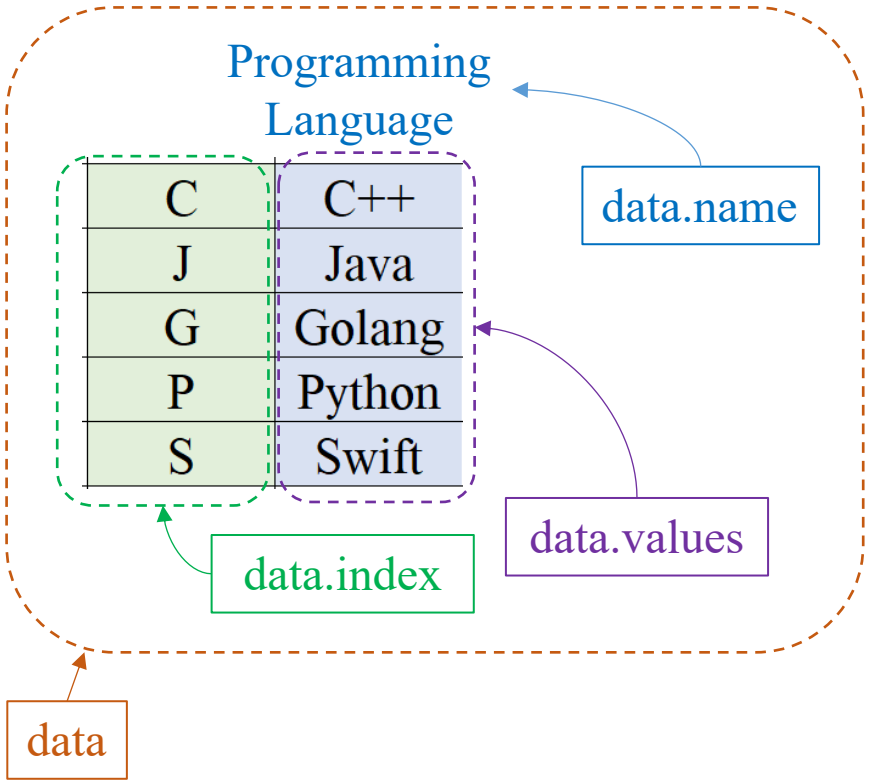
Series in Pandas

❖ Create a series

```python
import pandas as pd

data = pd.Series([5, 7, 5, 1, 6],
                 name='num_dropped')
```



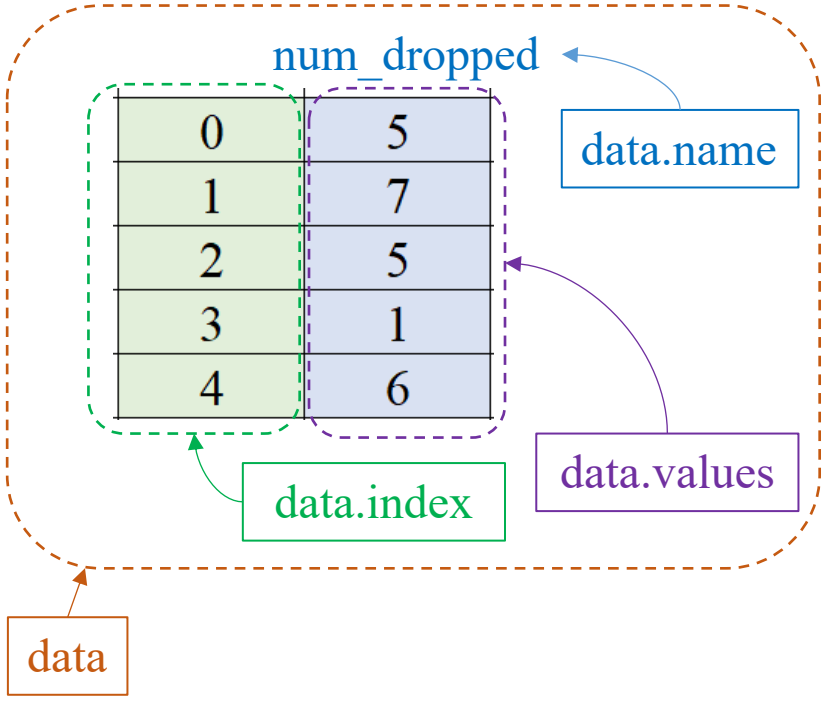

```python
import pandas as pd

data = pd.Series(['C++', 'Golang', 'Java', 'Python', 'Swift'],
                 index=list('CGJPS'),
                 name='Programming Language')
```



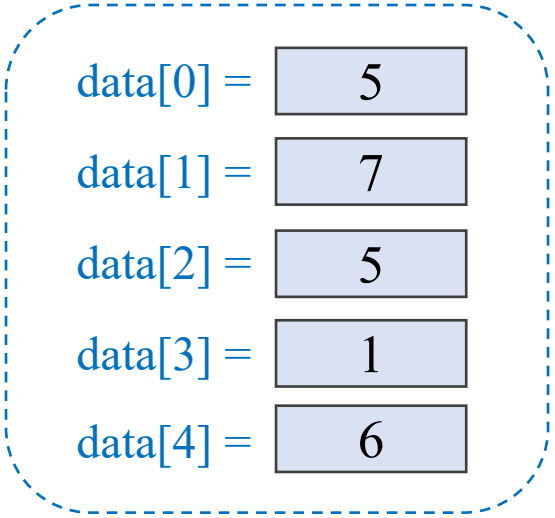
Series in Pandas

❖ Create a series

```
1  import pandas as pd
2
3  data = pd.Series([5, 7, 5, 1, 6],
4                   name='num_dropped')
```



❖ Get rows




```
for v in data:
    print(v)

#----------------

print(data[0])
print(data[1])
print(data[2])
print(data[3])
print(data[4])
```

```
print(data.loc[0])
print(data.loc[1])
print(data.loc[2])
print(data.loc[3])
print(data.loc[4])
```

result = data.loc[2:3]

result =

2	5
3	1

result = data[2:3]

result =

2	5

result = data[data.between(3, 6)]

result =

0	5
2	5
4	6

result = data[data>5]

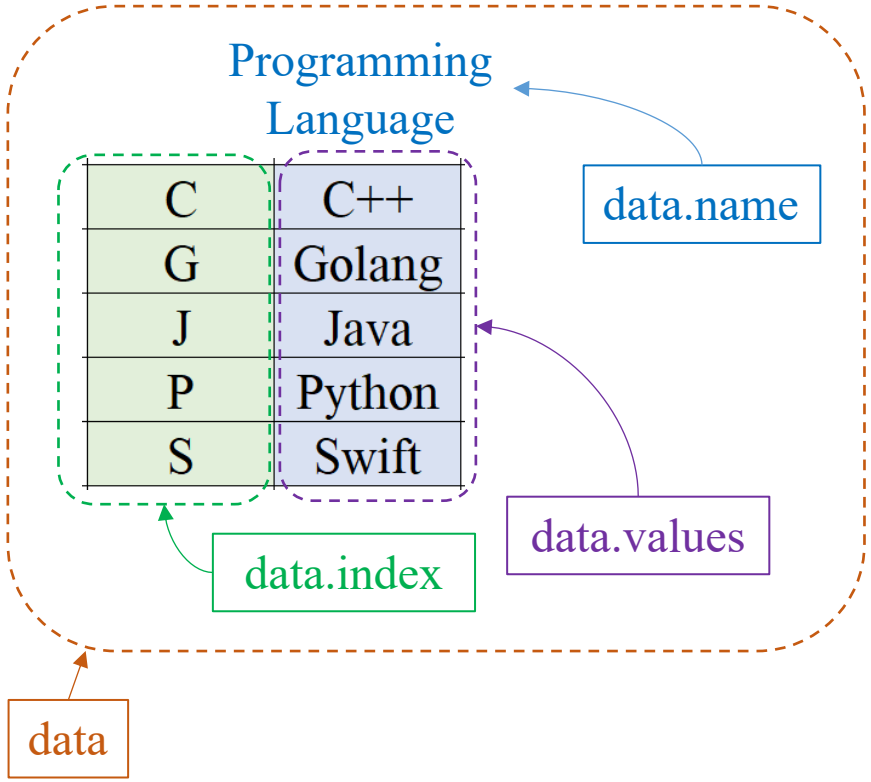
result =

1	7
4	6

Series in Pandas

❖ Create a series

```
1  import pandas as pd
2
3  data = pd.Series(['C++', 'Golang', 'Java', 'Python', 'Swift'],
4                   index=list('CGJPS'),
5                   name='Programming Language')
```



❖ Get rows

result = data['C':'P']

result =

C	C++
G	Golang
J	Java
P	Python

result = data[['C', 'P']]

result =

C	C++
P	Python

result = data[2:4]

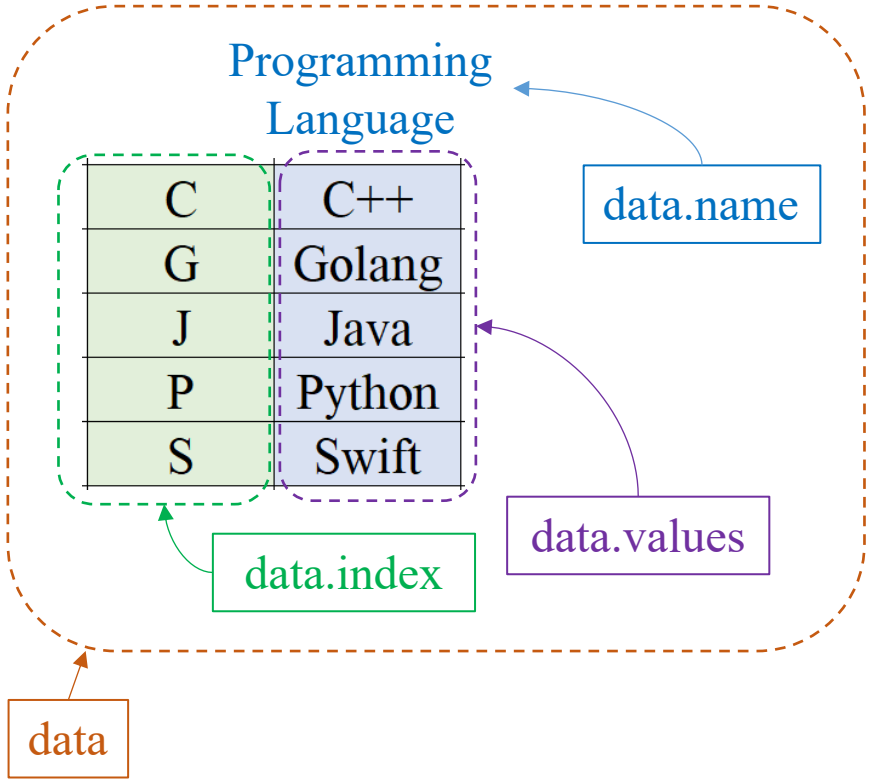
result =

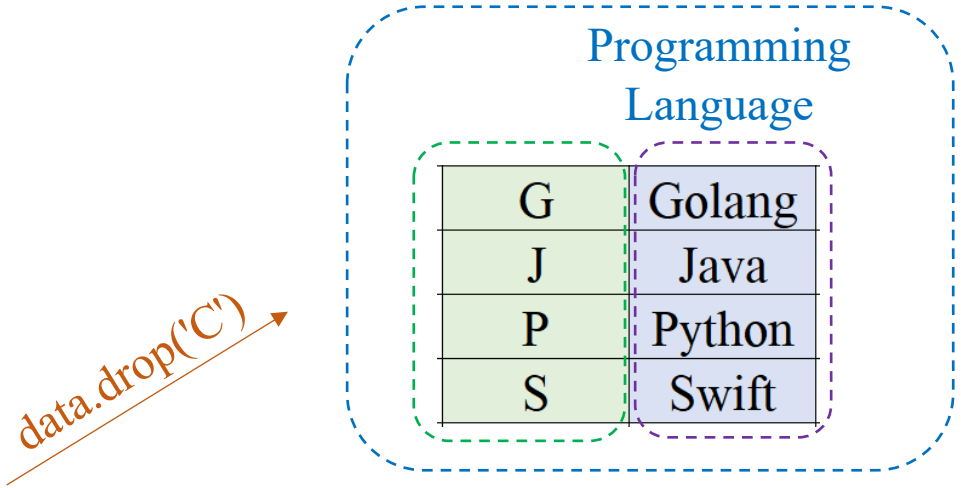
J	Java
P	Python

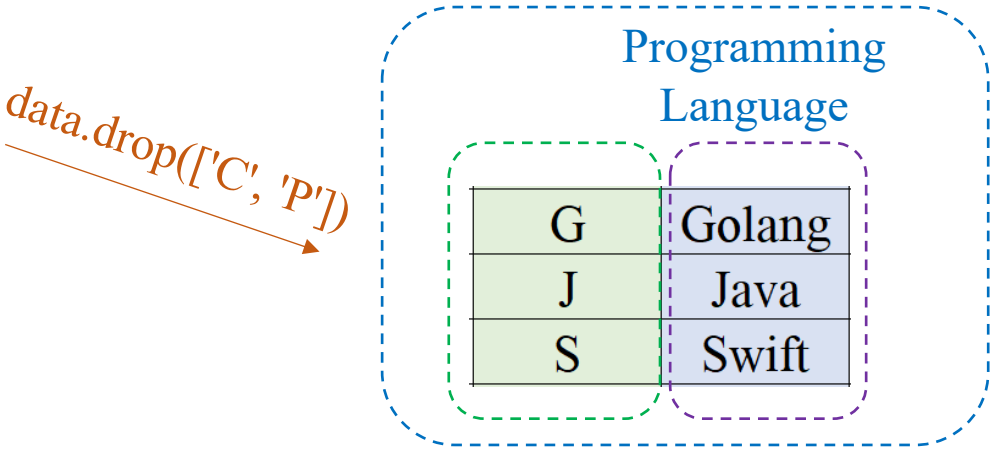
Series in Pandas

❖ Drop a row

```
1  import pandas as pd
2
3  data = pd.Series(['C++', 'Golang', 'Java', 'Python', 'Swift'],
4                   index=list('CGJPS'),
5                   name='Programming Language')
```



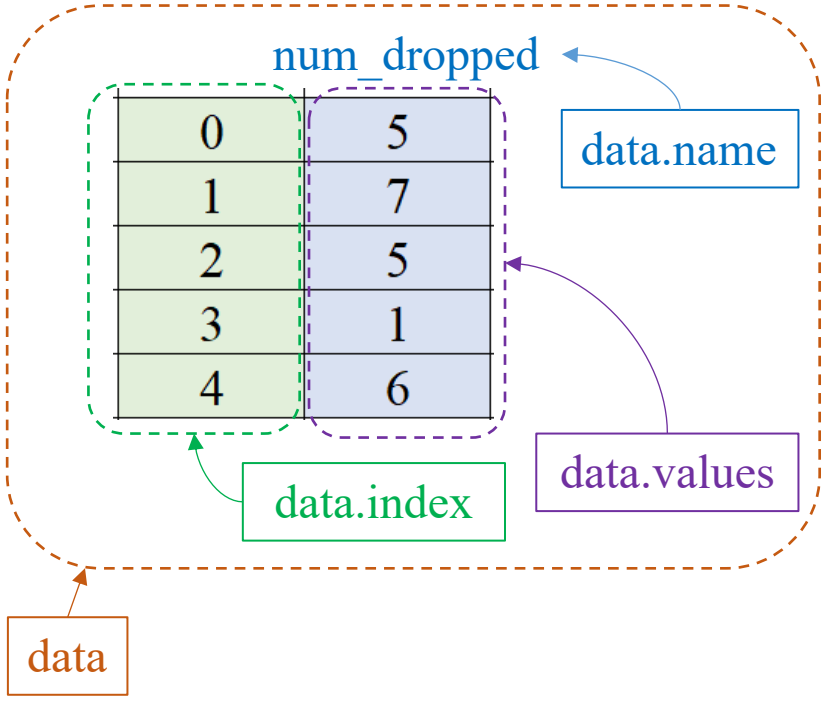




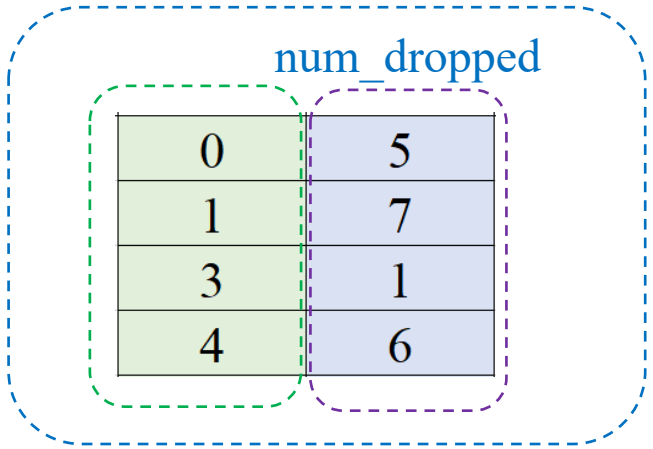
Series in Pandas

❖ Drop a row

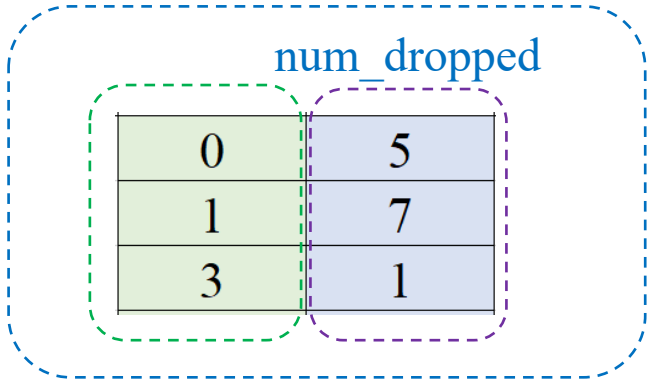
```
1  import pandas as pd
2
3  data = pd.Series([5, 7, 5, 1, 6],
4                   name='num_dropped')
```



data.drop(2)



data.drop([2, 4])



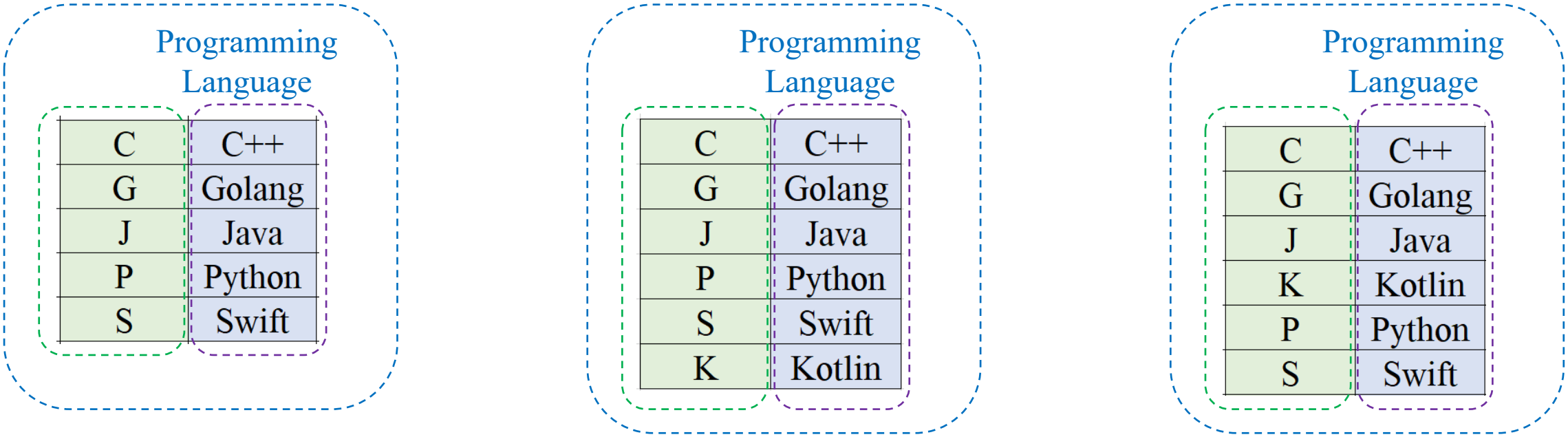
Series in Pandas

❖ Insert a row

```python
import pandas as pd

data = pd.Series(['C++', 'Golang', 'Java', 'Python', 'Swift'],
                 index=list('CGJPS'),
                 name='Programming Language')
```

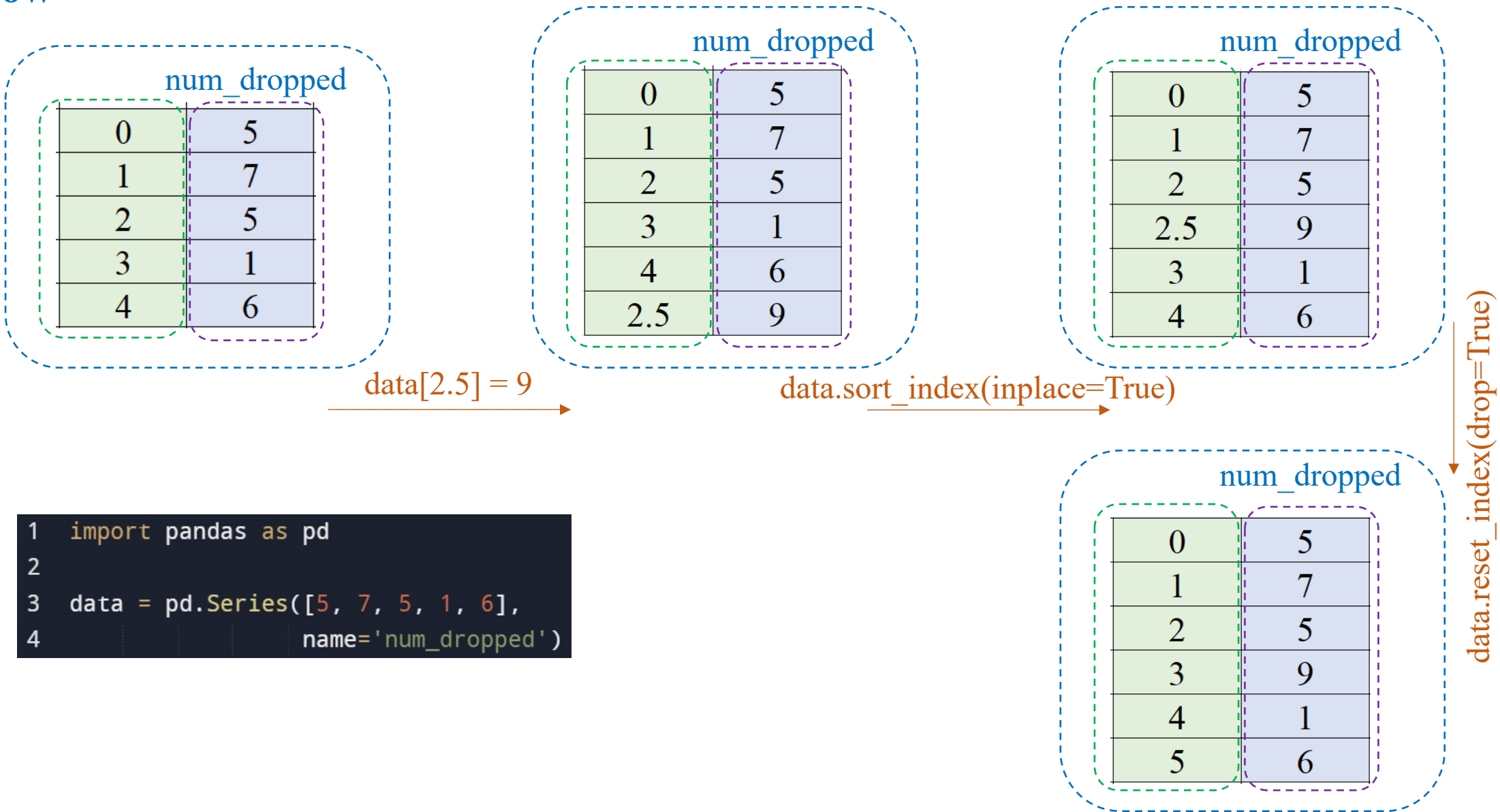
data['K'] = 'Kotlin '

data.sort_index(inplace=True)



Series in Pandas

❖ Insert a row

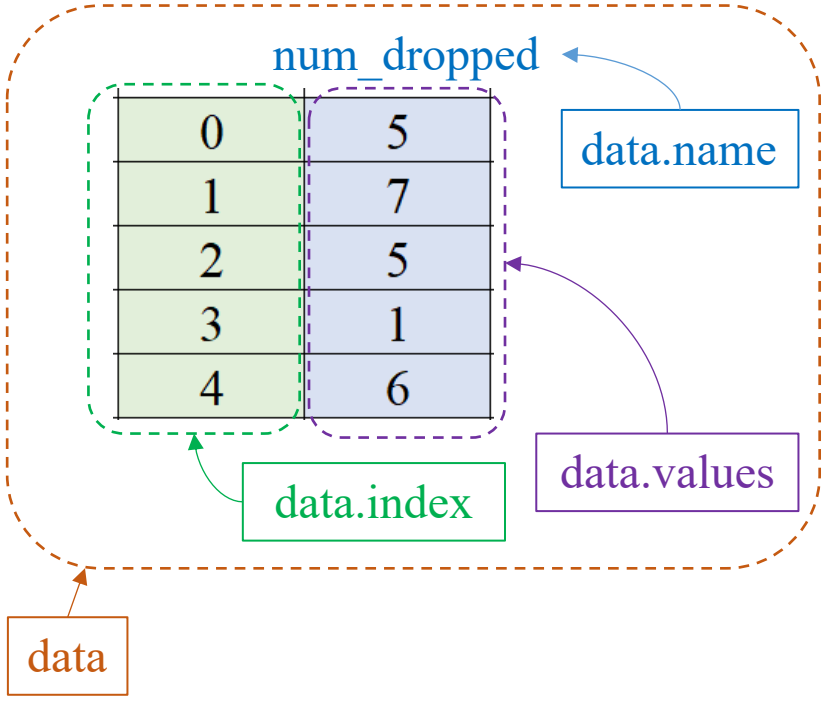



```
1  import pandas as pd
2
3  data = pd.Series([5, 7, 5, 1, 6],
4                   name='num_dropped')
```

Series in Pandas

❖ Common Pandas functions

```
1  import pandas as pd
2
3  data = pd.Series([5, 7, 5, 1, 6],
4                   name='num_dropped')
```



data.min() → 1

data.sum() → 24

data.max() → 7

data.mean() → 4.8

data.std() → 2.28

data.var() → 5.2

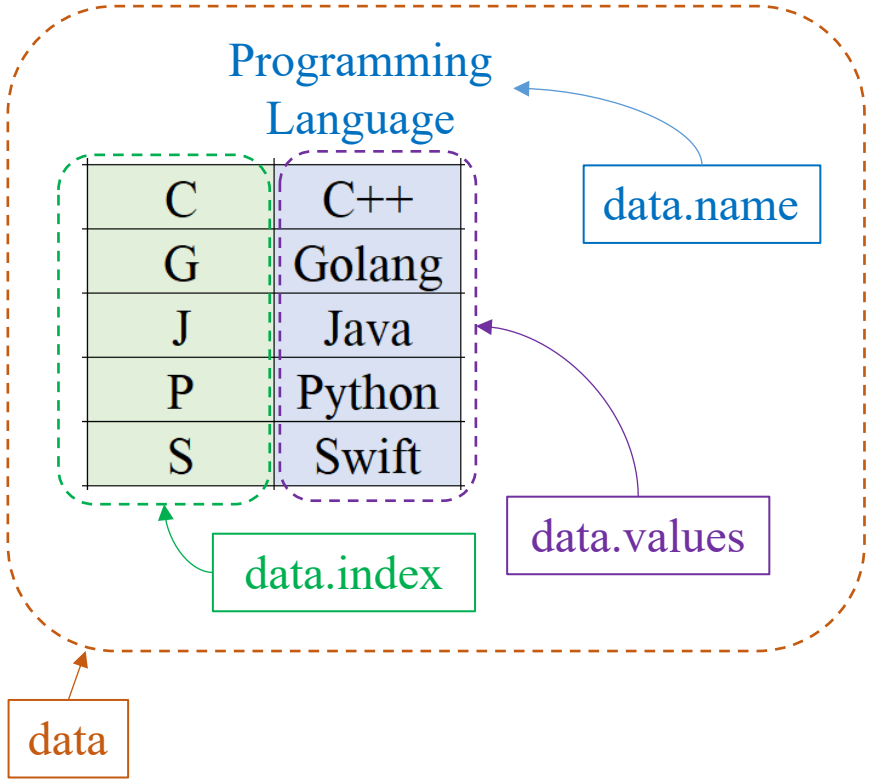
data.idxmax() → 1

data.argmax() → 1

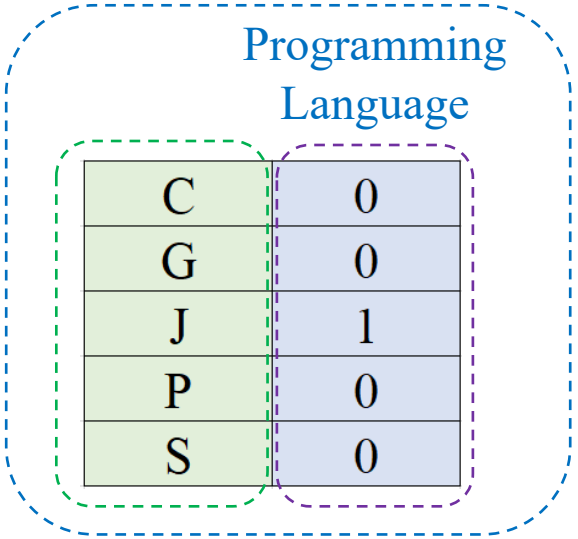
Series in Pandas

❖ Common Pandas functions

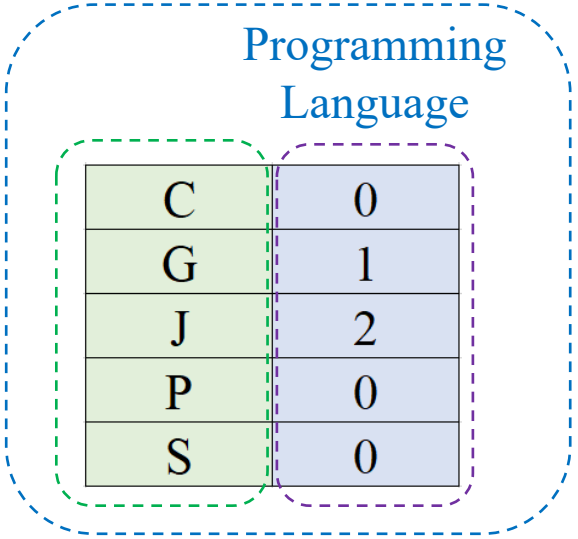
```
1  import pandas as pd
2
3  data = pd.Series(['C++', 'Golang', 'Java', 'Python', 'Swift'],
4                   index=list('CGJPS'),
5                   name='Programming Language')
```



data.str.count('Java')



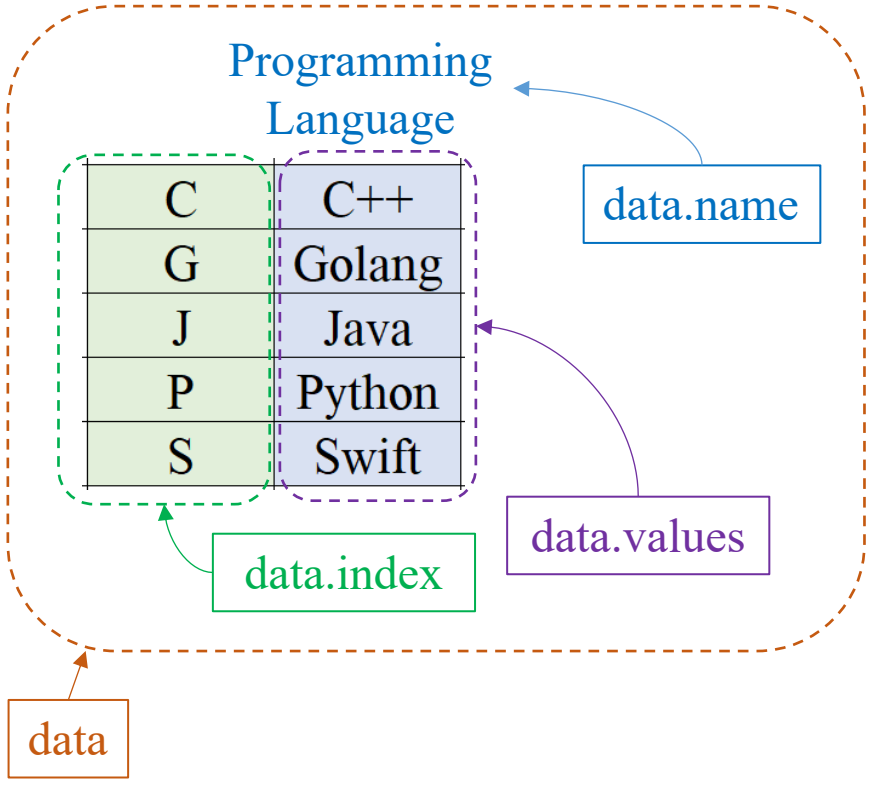
data.str.count('a')

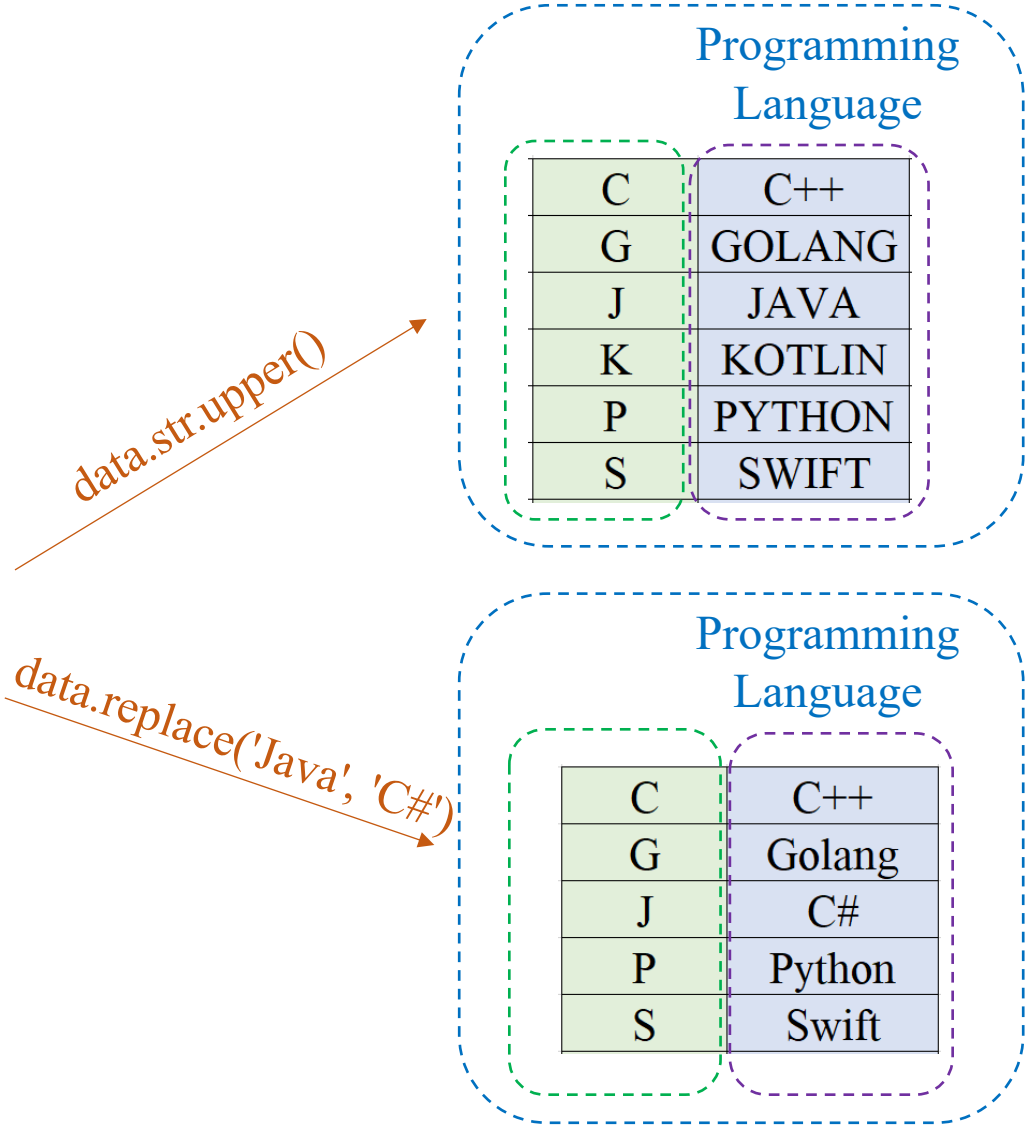


Series in Pandas

❖ Common Pandas functions

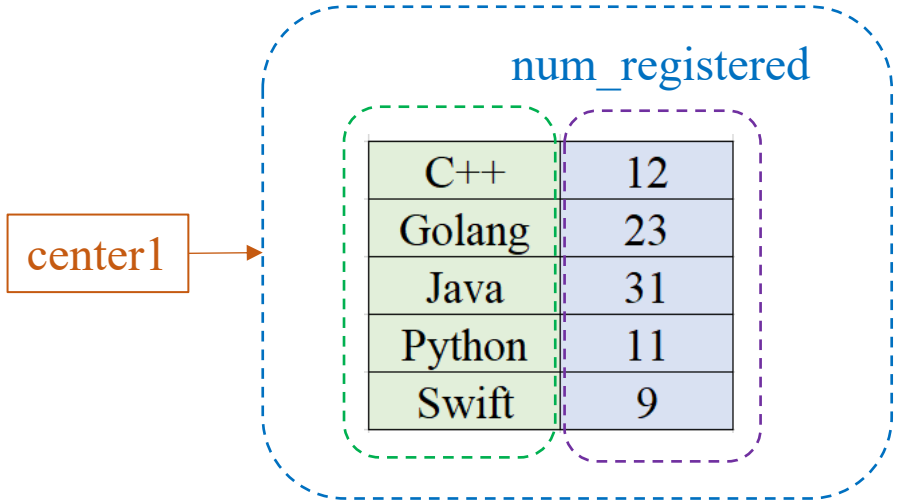
```
1  import pandas as pd
2
3  data = pd.Series(['C++', 'Golang', 'Java', 'Python', 'Swift'],
4                   index=list('CGJPS'),
5                   name='Programming Language')
```



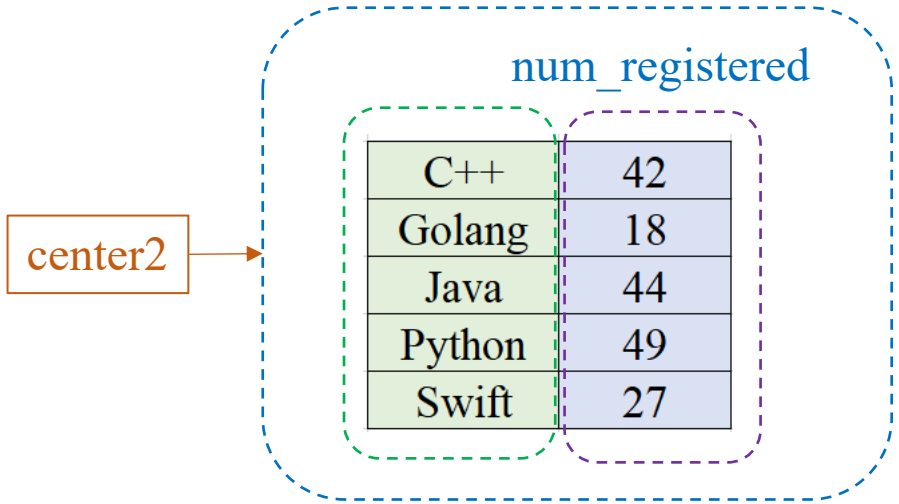


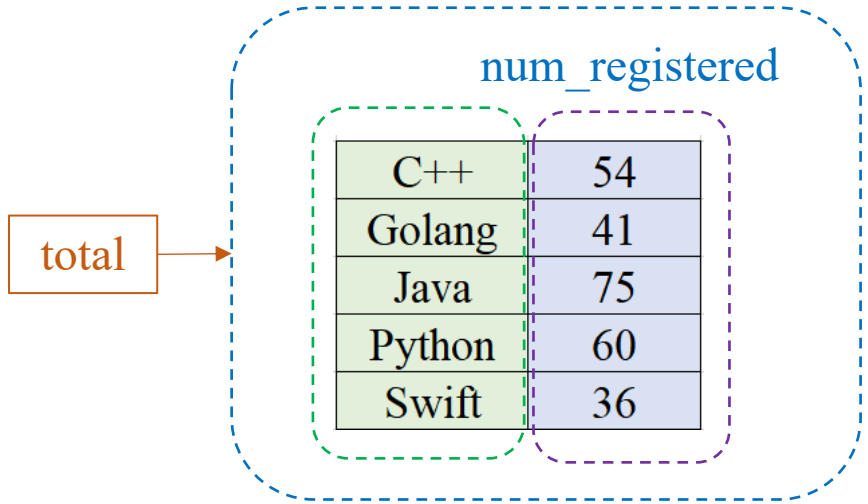
Series in Pandas

Addition between two series



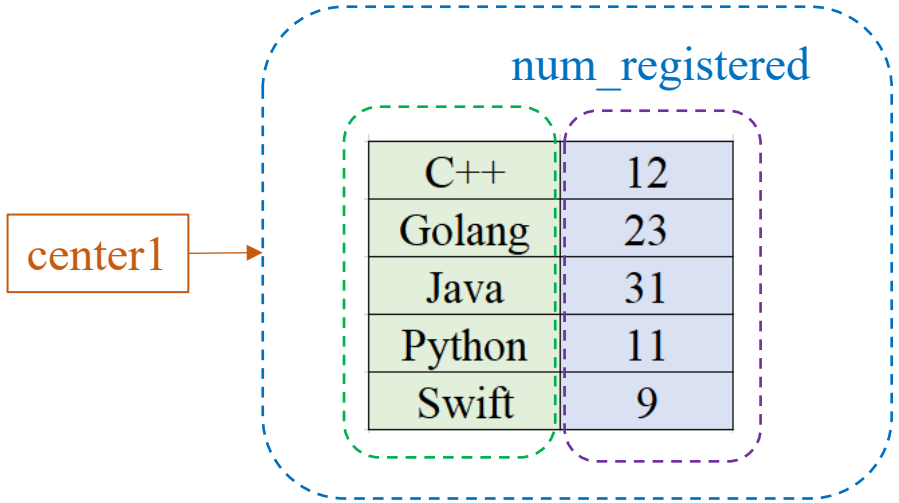

```python
import pandas as pd

center1 = pd.Series([12, 23, 31, 11, 9],
                    index=list(['C++', 'Golang',
                                'Java', 'Python',
                                'Swift']),
                    name='num_registered')

center2 = pd.Series([42, 18, 44, 49, 27],
                    index=list(['C++', 'Golang',
                                'Java', 'Python',
                                'Swift']),
                    name='num_registered')

total = center1 + center2
```



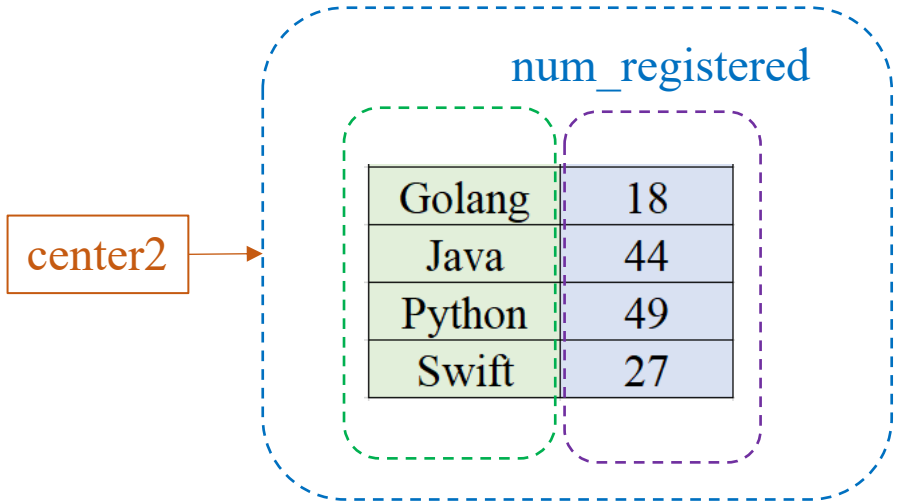


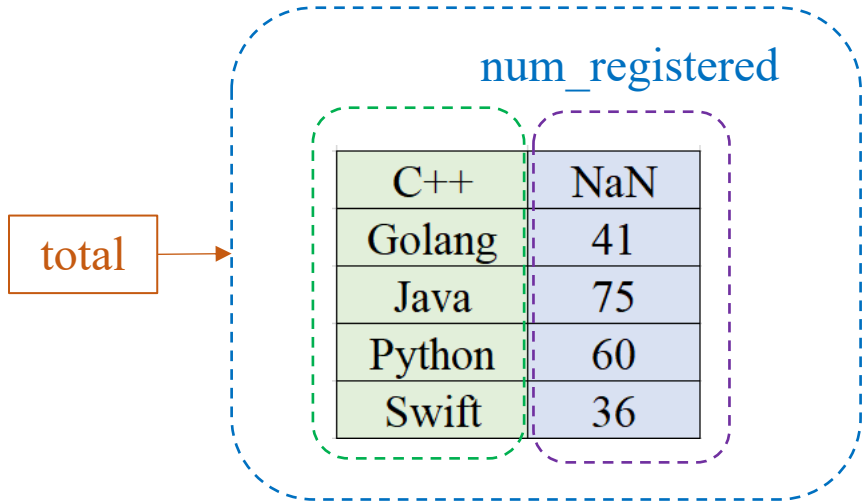
Series in Pandas

Addition between two series




```python
import pandas as pd

center1 = pd.Series([12, 23, 31, 11, 9],
                    index=list(['C++', 'Golang',
                                'Java', 'Python',
                                'Swift']),
                    name='num_registered')

center2 = pd.Series([18, 44, 49, 27],
                    index=list(['Golang', 'Java',
                                'Python', 'Swift']),
                    name='num_registered')

total = center1 + center2
```





Series in Pandas

❖ Group values

```
1  import pandas as pd
2
3  data = pd.Series([4, 1, 0, 2, 2, 1],
4                   index=list(['C++', 'Golang',
5                               'Java', 'Golang',
6                               'Java', 'C++']),
7                   name='num_dropped')
```

data.groupby(level=0).sum()

num_dropped

C++	5
Golang	3
Java	2

data.groupby(data>3).sum()

num_dropped

FALSE	6
TRUE	4

num_dropped

C++	4
Golang	1
Java	0
Golang	2
Java	2
C++	1

data.name

data.values

data.index

data

data['C++']

C++	4
C++	1

data['C++'].sum() → 5

data['Golang']

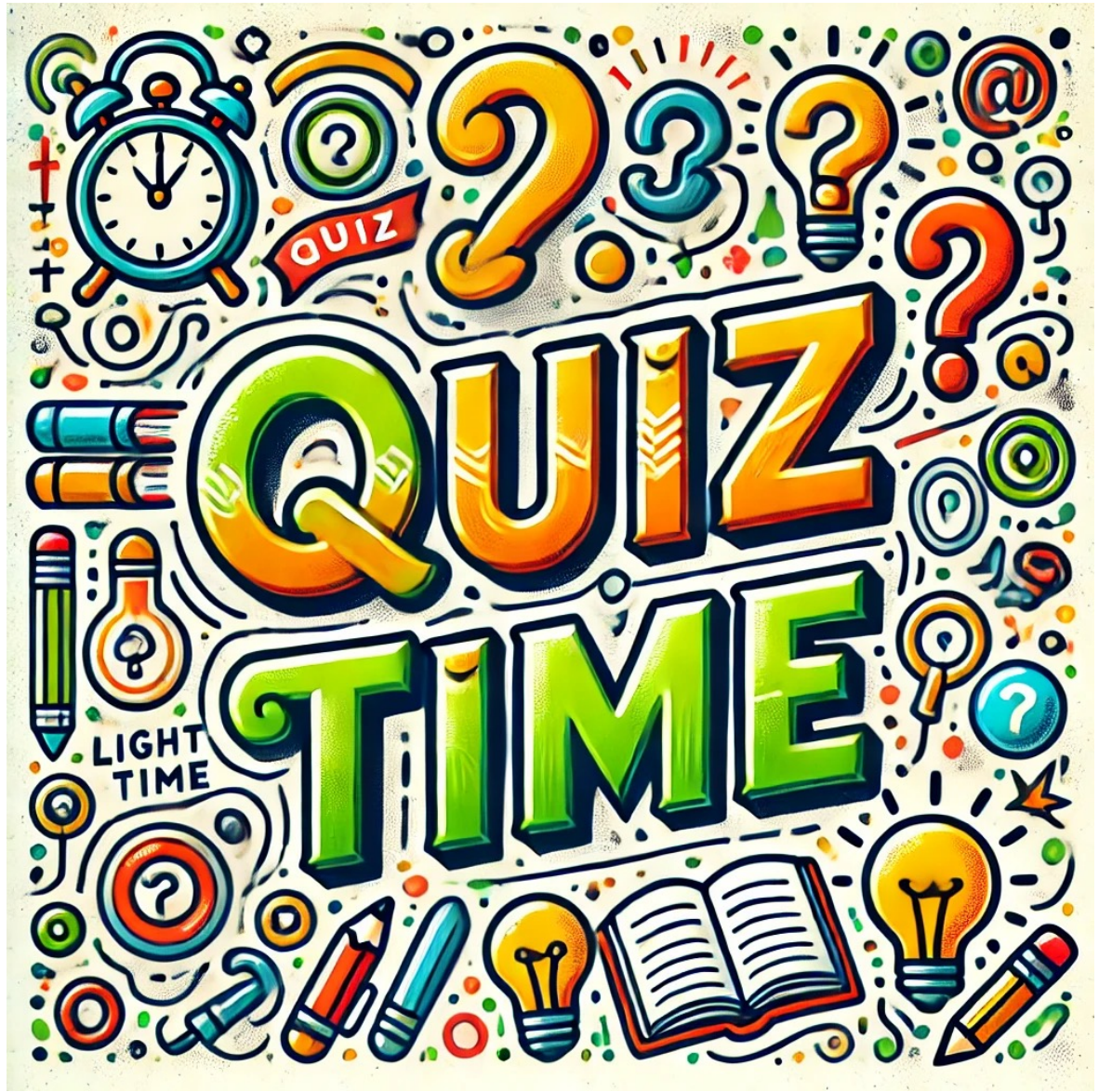
Golang	1
Golang	2

data['Golang'].sum() → 3

data['Java']

Java	0
Java	2

data['Java'].sum() → 2


QUIZ
QUIZ
TIME
LIGHT
TIME

Series in Pandas

❖ Quiz

```
import pandas as pd

data = pd.Series([5, 7, 1],
                 name='num_dropped')
print(data[[1, 2]])
```

```
import pandas as pd

data = pd.Series([5, 7, 1],
                 name='num_dropped')
print(data[[True, False, True]])
```

```
import pandas as pd

data = pd.Series([5, 7, 1],
                 name='num_dropped')
print(data[[True, False]])
```

```
import pandas as pd

data = pd.Series([5, 7, 1, 2],
                 name='num_dropped')
print(data.groupby(data > 3).sum())
```

```
import pandas as pd

data = pd.Series([5, 7, 1, 2],
                 name='num_dropped')
print(data.groupby([True, False, True, True]).sum())
```

Series in Pandas

❖ Quiz

```
data = pd.Series(['Java', 'C++', 'Swift'],
                 index=list('JCS'),
                 name='Programming Language')
```

A1

```
data_sorted = data.sort_index(inplace=True)
print(data_sorted)
```

A2

```
data_sorted = data.sort_index(inplace=False)
print(data_sorted)
```

Outline

SECTION 1

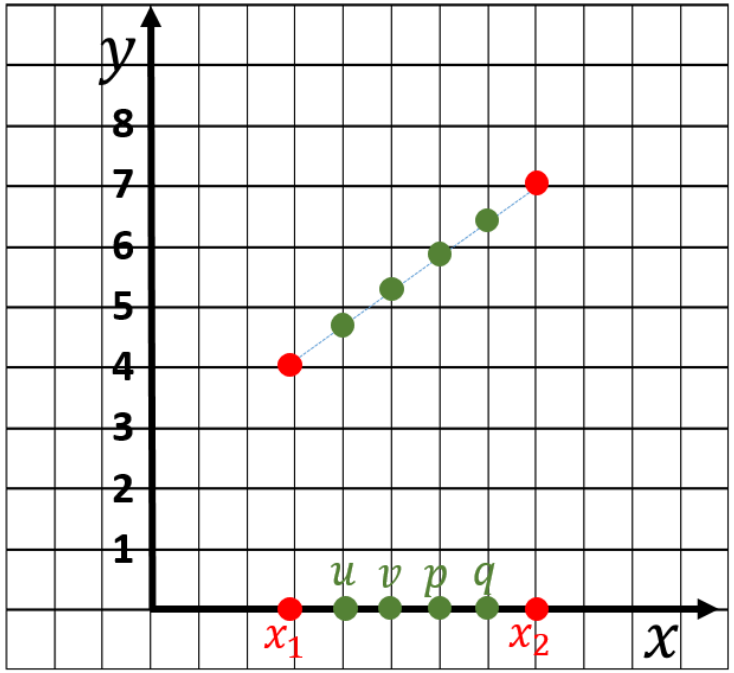
Series in Pandas

SECTION 2

Operations

SECTION 3

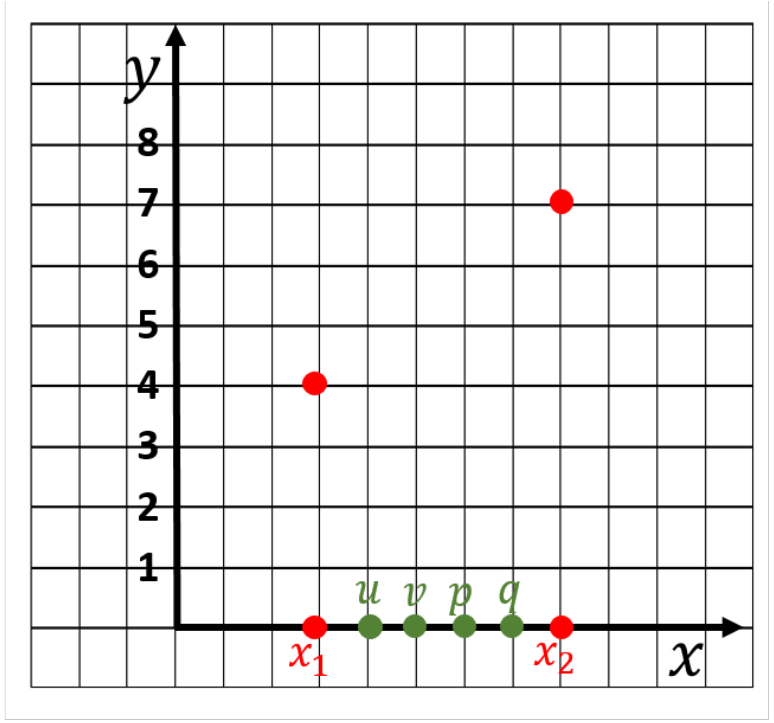
Case Studies



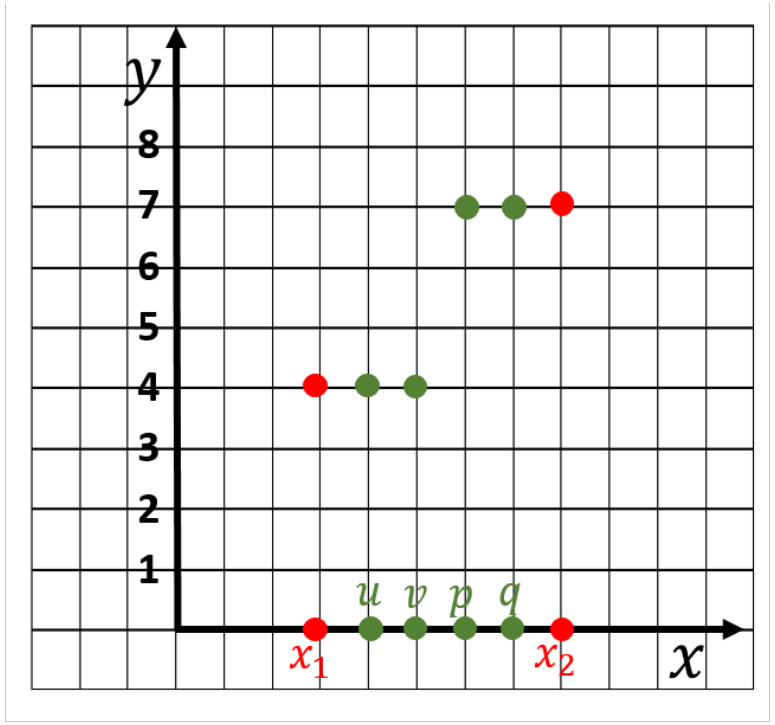
Nội suy theo hàm tuyến tính

1D Interpolation

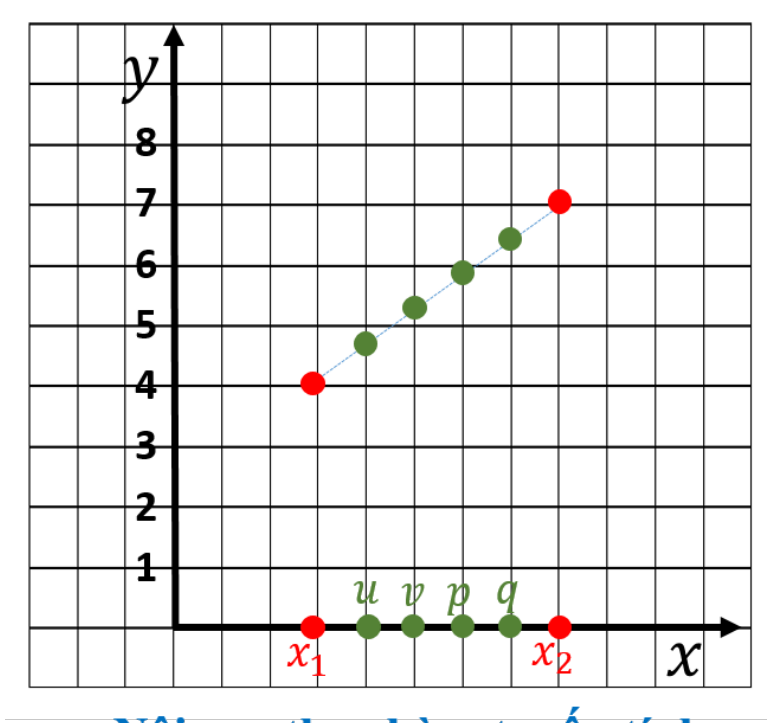
❖ Two simplest techniques



Tìm giá trị cho các vị trí u, v, p và q



Nearest neighbor: Tính khoảng cách đến x_1 và x_2, và lấy giá trị của x gần hơn



Nội suy theo hàm tuyến tính

Given $A(x_1, y_1)$ and $B(x_2, y_2)$

Line equation: $y = ax + b$ (1)

From A and B, we have $a = \frac{y_2 - y_1}{x_2 - x_1}$ (2)

Replace A into (1), we have $b = y_1 - \frac{y_2 - y_1}{x_2 - x_1} x_1$ (3)

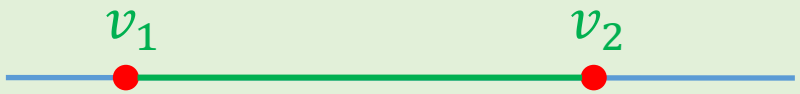
From (1), (2), and (3) $\Rightarrow y = \frac{y_2 - y_1}{x_2 - x_1} x + \left(y_1 - \frac{y_2 - y_1}{x_2 - x_1} x_1 \right)$ (4)

$$(4) \Rightarrow y = \frac{(y_2 - y_1)x}{x_2 - x_1} + \frac{y_1(x_2 - x_1) - x_1(y_2 - y_1)}{x_2 - x_1}$$

$$= \frac{y_2 x - y_1 x + y_1 x_2 - y_1 x_1 - y_2 x_1 + y_1 x_1}{x_2 - x_1}$$

$$= \underbrace{\frac{x_2 - x}{x_2 - x_1}}_{\alpha} y_1 + \underbrace{\frac{x - x_1}{x_2 - x_1}}_{1 - \alpha} y_2$$

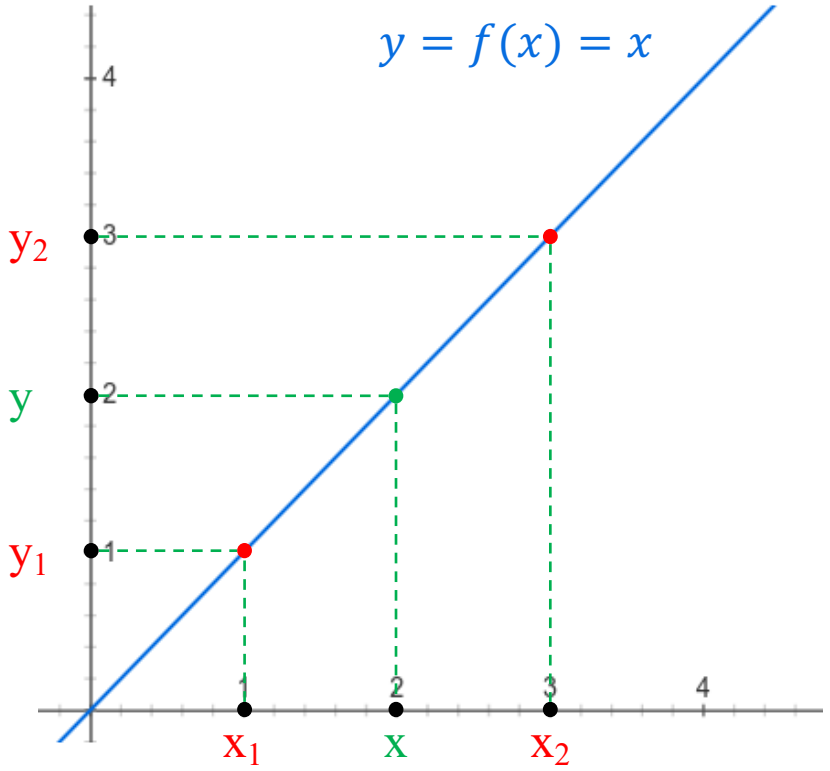
$$\frac{x_2 - x}{x_2 - x_1} + \frac{x - x_1}{x_2 - x_1} = 1$$



Line segment equation

$$v = \alpha v_1 + (1 - \alpha) v_2$$

Linear interpolation



Giả xử không biết đường thẳng $y = x$, chỉ biết hai điểm điểm màu đỏ (x_1=1, y_1=1) và (x_2=3, y_2=3). Tìm điểm màu xanh lá có y=? khi biết x=2.

$$y \approx \underbrace{\frac{x_2 - x}{x_2 - x_1} y_1}_{\text{Tỉ lệ y phụ thuộc vào } y_1} + \underbrace{\frac{x - x_1}{x_2 - x_1} y_2}_{\text{Tỉ lệ y phụ thuộc vào } y_2} = \frac{3-2}{3-1} * 1 + \frac{2-1}{3-1} * 3 = 2$$

(x=1.1, y=?) $y \approx \frac{3-1.1}{3-1} * 1 + \frac{1.1-1}{3-1} * 3 \approx 0.95 * 1 + 0.05 * 3 \approx 1.1$

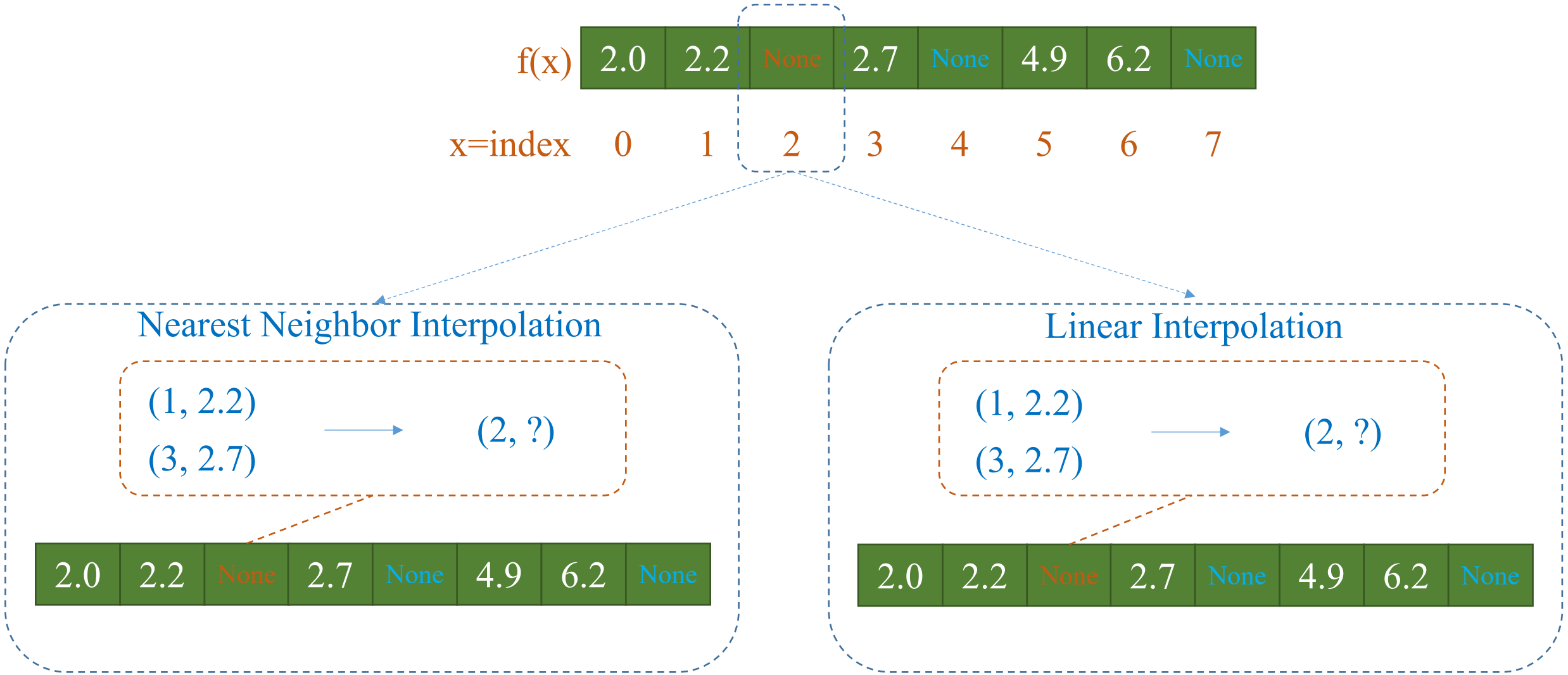
Càng gần x_1, y phụ thuộc nhiều vào y_1 và ít phụ thuộc y_2

(x=2.9, y=?) $y \approx \frac{3-2.9}{3-1} * 1 + \frac{2.9-1}{3-1} * 3 \approx 0.05 * 1 + 0.95 * 3 \approx 2.9$

Càng gần x_2, y phụ thuộc nhiều vào y_2 và ít phụ thuộc y_1

Application to Time-Series Data

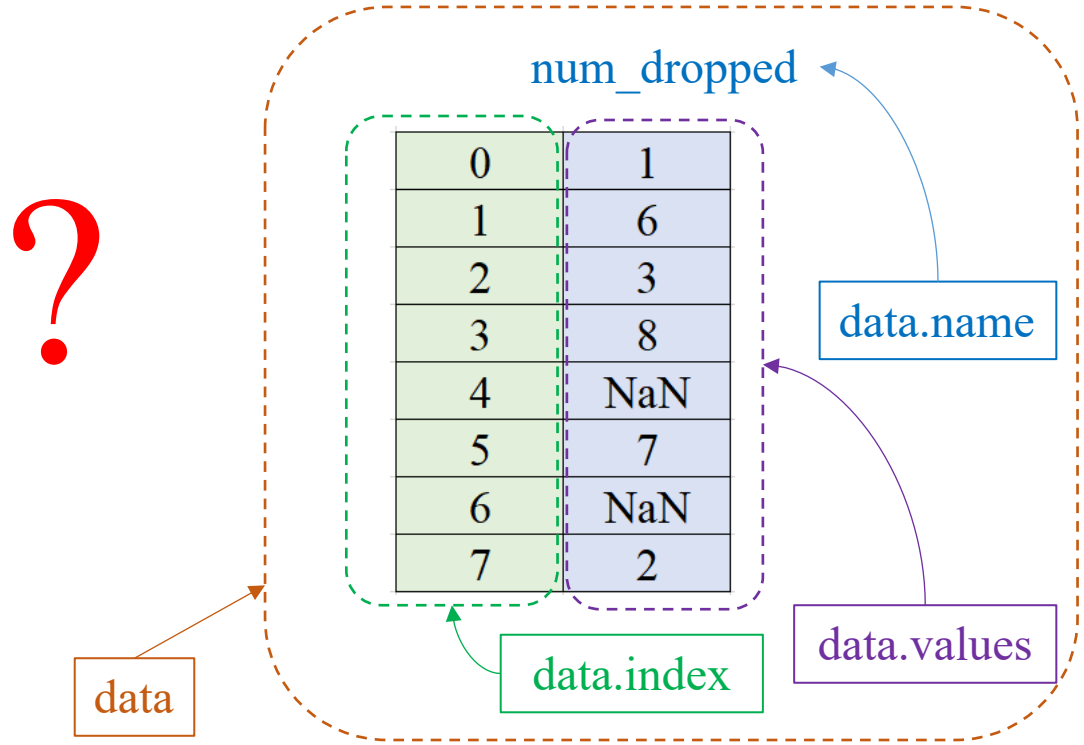
❖ Quiz



Series in Pandas

❖ Missing values

```
1  import pandas as pd
2
3  data = pd.Series([1, 6, 3, 8, np.nan, 7, np.nan, 2],
4                   name='num_dropped')
```



data.min() → 1

data.sum() → 27

data.max() → 8

data.mean() → 4.5

data.std() → 2.88

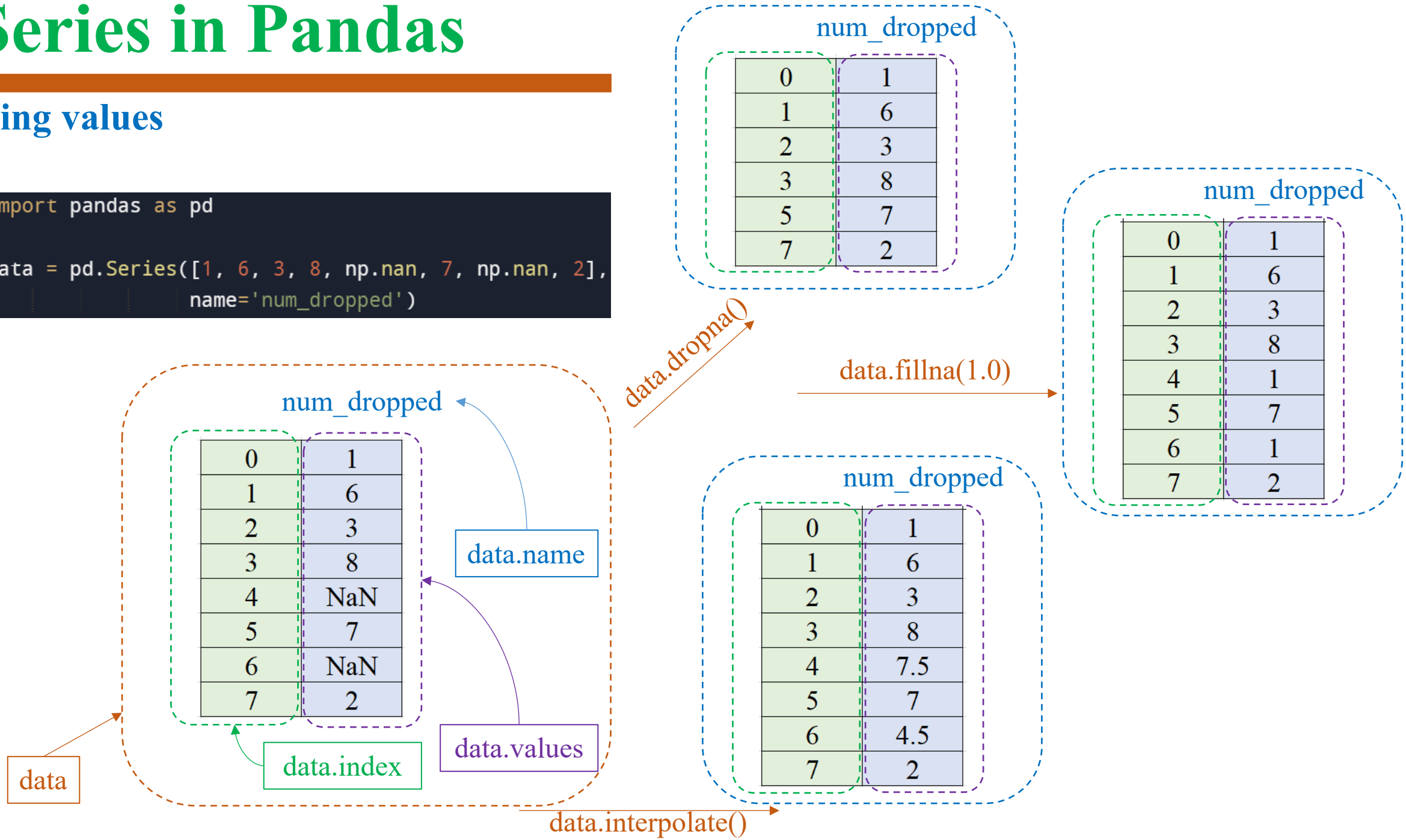
data.idxmax() → 3

data.argmax() → 3

Series in Pandas

❖ Missing values

```
1  import pandas as pd
2
3  data = pd.Series([1, 6, 3, 8, np.nan, 7, np.nan, 2],
4                   name='num_dropped')
```



Percentile and BoxPlot

Median

❖ Definition

Data

$$X = \{X_1, \dots, X_N\}$$

Formula

Step 1: Sort X → S

Step 2

If N is odd, then $m = S_{\left(\frac{N+1}{2}\right)}$

If N is even, then $m = \left(S_{\left(\frac{N}{2}\right)} + S_{\left(\frac{N}{2}+1\right)}\right)/2$

Given the data

$$X = \{2, 8, 5, 4, 1\}$$

$$N = 5$$

Step 1

$$S = \{1, 2, 4, 5, 8\}$$

1 2 3 4 5

Step 2; $N = 5$

$$k = \frac{N+1}{2} = 3$$

$$m = S_k = 4$$

Median

❖ Definition

Data

$$X = \{X_1, \dots, X_N\}$$

Formula

Step 1: Sort X → S

Step 2

If N is odd, then $m = S_{\left(\frac{N+1}{2}\right)}$

If N is even, then $m = \left(S_{\left(\frac{N}{2}\right)} + S_{\left(\frac{N}{2}+1\right)}\right)/2$

Given the data

$$X = \{2, 8, 5, 4, 1, 8\}$$

$$N = 6$$

Step 1

$$S = \{1, 2, 4, 5, 8, 8\}$$
$$1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$$

Step 2; $N = 6$

$$m = \frac{S_3 + S_4}{2}$$

$$= \frac{4 + 5}{2} = 4.5$$

Median

❖ Quiz

data1 = [5, 5, 7, 8, 9, 10, 12, 14, 15, 17]

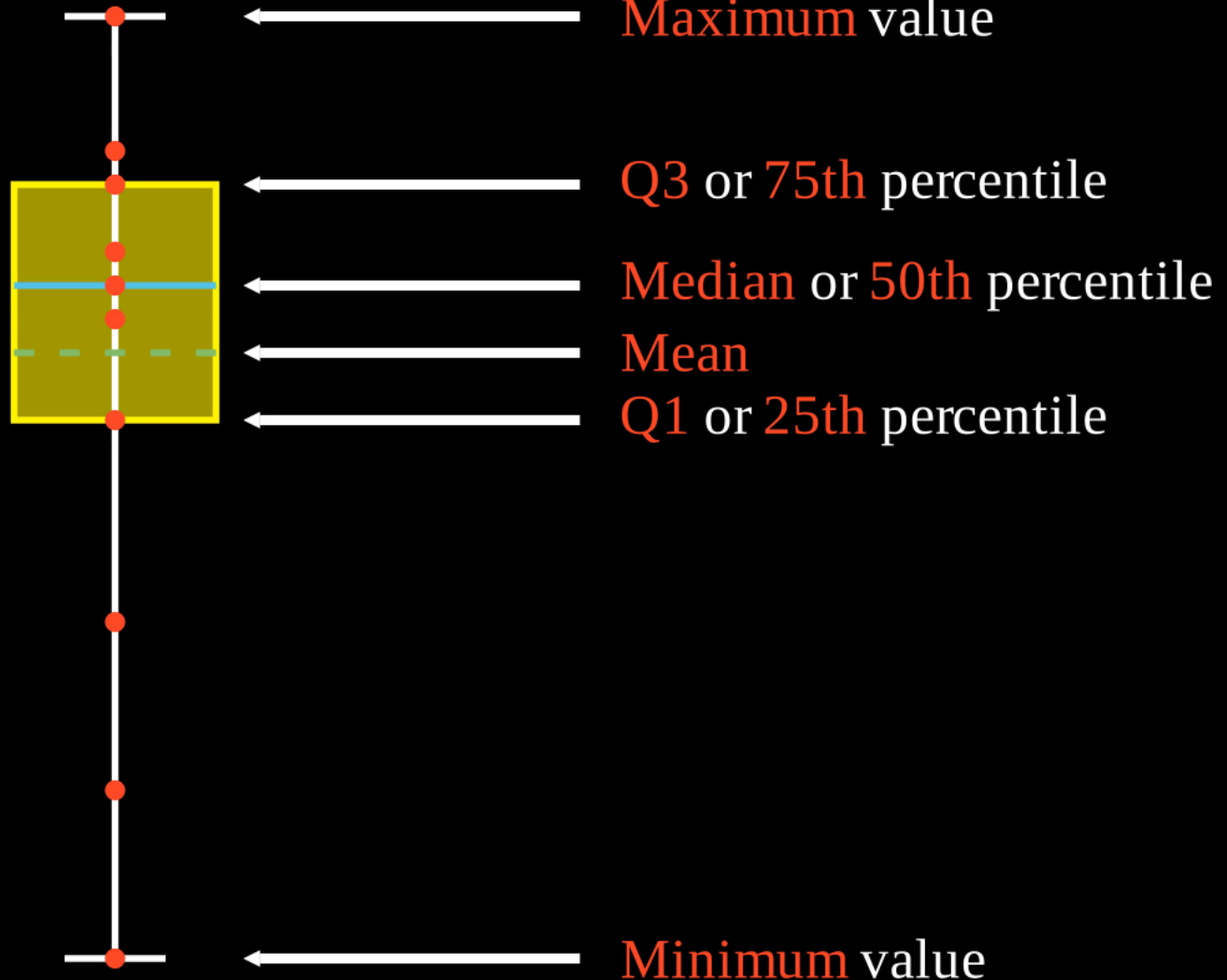
data2 = [5, 5, 7, 8, 9, 10, 12, 14, 15, 17, 18]

AI VIETNAM

AI VIETNAM

But what is a box plot?

Graphically display various parameters



To understand what these parameters mean

first we need to understand what is

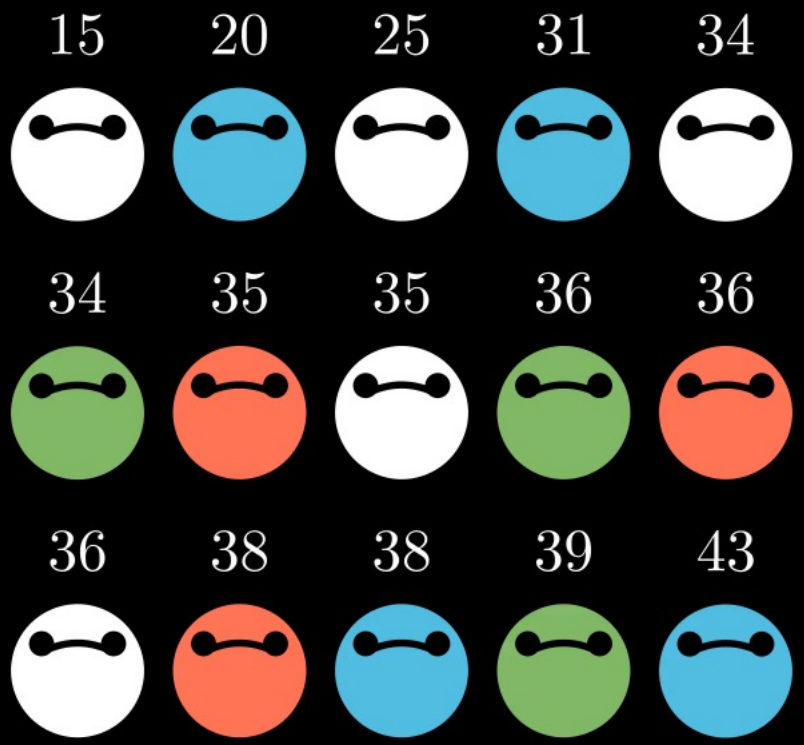
Percentile

AI VIETNAM

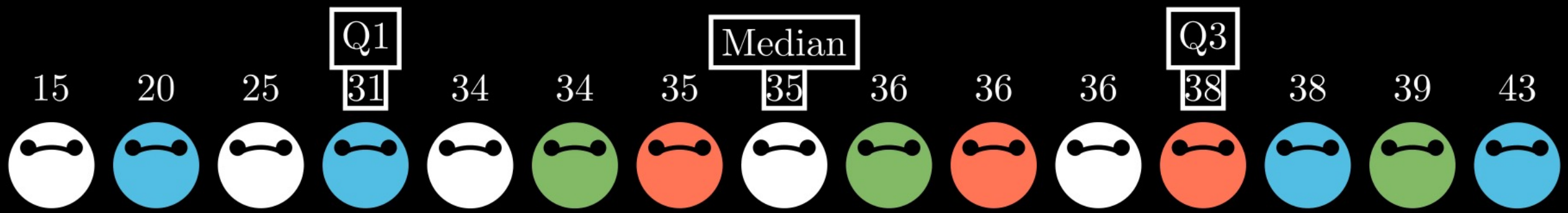
TEST SCORES OF 15 STUDENTS

CLASS SIZE

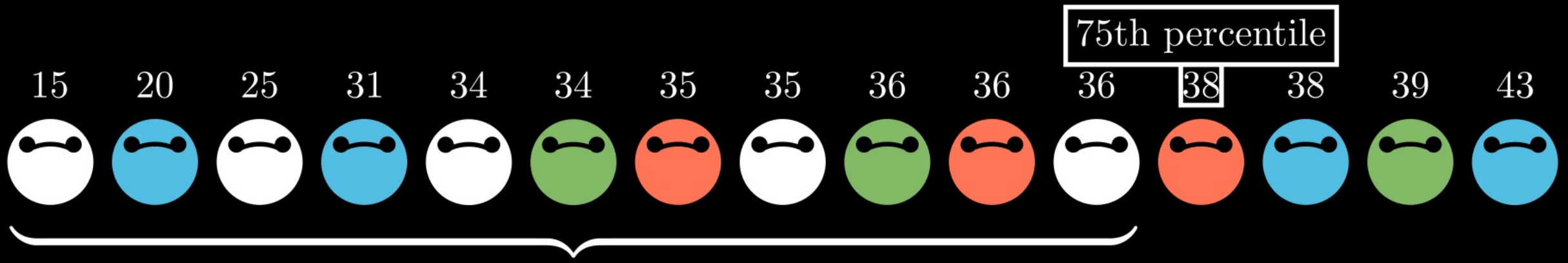
n = 15



AI VIETNAM



AI VIETNAM



beat 75% students

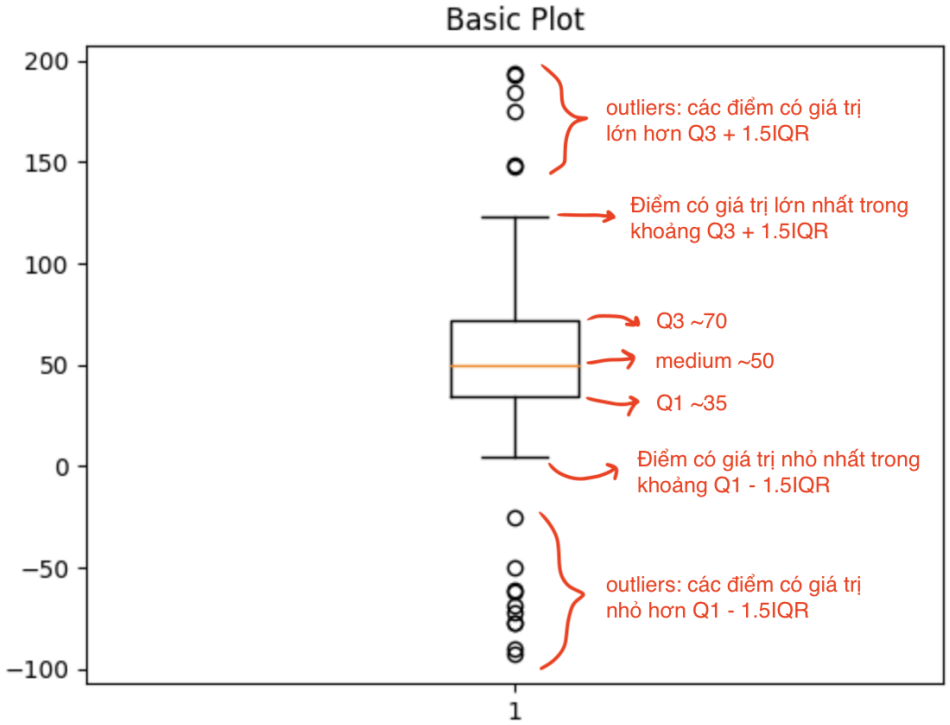
Boxplot

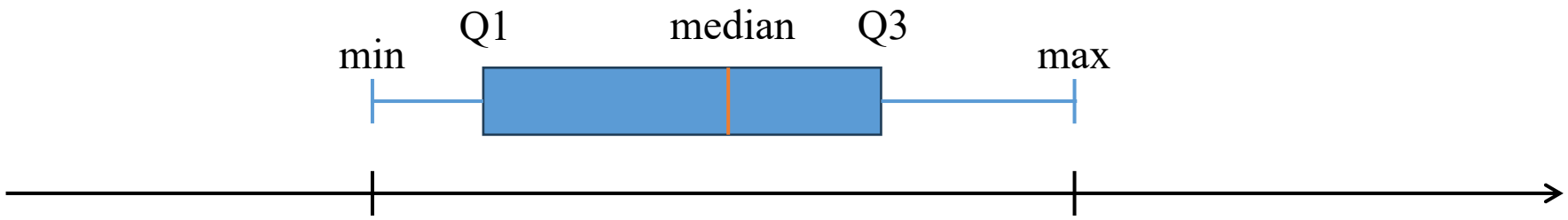
❖ Create a boxplot

data = [5, 5, 7, 8, 9, 10, 12, 14, 15, 17]

Compute: Min, Max, Median, Q1, and Q3

1) Ascending order
2) Find median
3) Find Q1 and Q3
4) Interquatile range IQR = Q3 – Q1





Outline

SECTION 1

Series in Pandas

SECTION 2

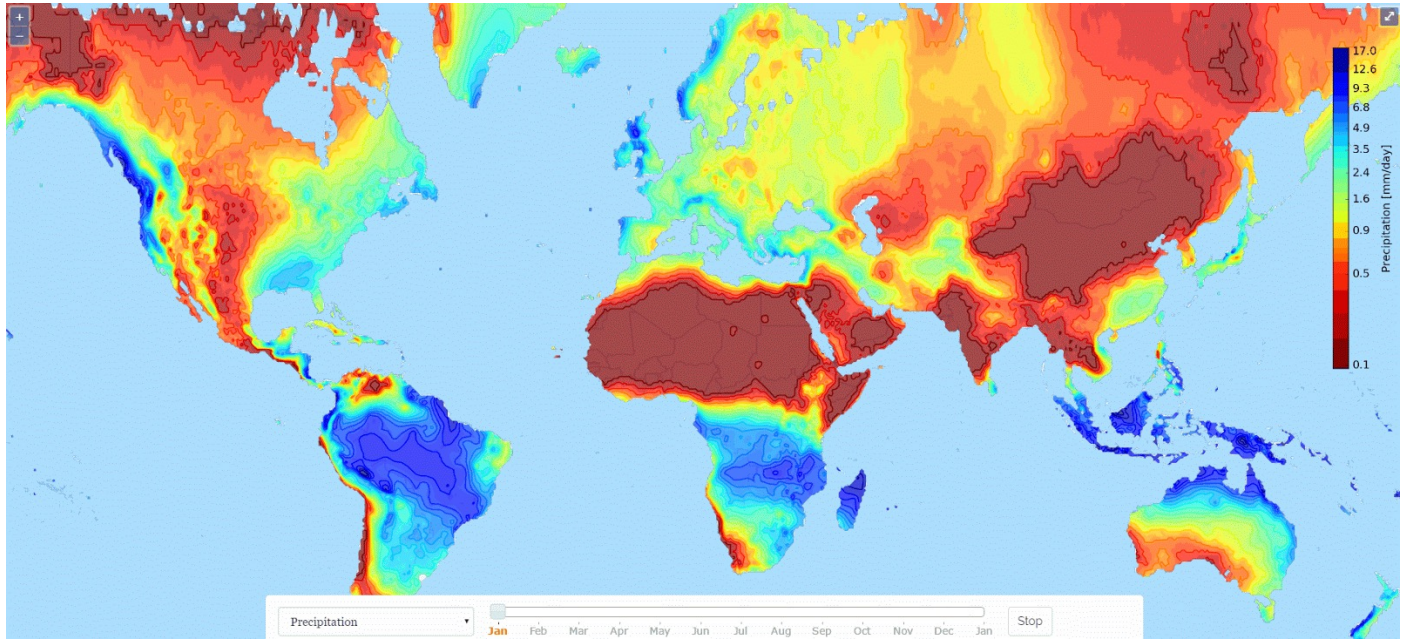
Operations

SECTION 3

Case Studies

Case Study

❖ Temperature forecasting



Predict future temperature in weather forecasting

Case Study

❖ Temperature forecasting

Problem Statement: Given temperature from the previous 6 hours (including the current one), predict temperature of the next 1 hour.

Hour	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00
Condition								
Temperature	32	31	31	30	29	26	25	

Case Study

Temperature forecasting

Date	Summary	Precip Type	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	Wind Bearing (degrees)	Visibility (km)
2006-04-01 00	Partly Cloudy	rain	9.472222222	7.388888889	0.89	14.1197	251	15.8263
2006-04-01 01	Partly Cloudy	rain	9.355555556	7.227777778	0.86	14.2646	259	15.8263
2006-04-01 02	Mostly Cloudy	rain	9.377777778	9.377777778	0.89	3.9284	204	14.9569
2006-04-01 03	Partly Cloudy	rain	8.288888889	5.944444444	0.83	14.1036	269	15.8263
2006-04-01 04	Mostly Cloudy	rain	8.755555556	6.977777778	0.83	11.0446	259	15.8263
2006-04-01 05	Partly Cloudy	rain	9.222222222	7.111111111	0.85	13.9587	258	14.9569
2006-04-01 06	Partly Cloudy	rain	7.733333333	5.522222222	0.95	12.3648	259	9.982
2006-04-01 07	Partly Cloudy	rain	8.772222222	6.527777778	0.89	14.1519	260	9.982
2006-04-01 08	Partly Cloudy	rain	10.82222222	10.82222222	0.82	11.3183	259	9.982
2006-04-01 09	Partly Cloudy	rain	13.77222222	13.77222222	0.72	12.5258	279	9.982
2006-04-01 10	Partly Cloudy	rain	16.01666667	16.01666667	0.67	17.5651	290	11.2056
2006-04-01 11	Partly Cloudy	rain	17.14444444	17.14444444	0.54	19.7869	316	11.4471
2006-04-01 12	Partly Cloudy	rain	17.8	17.8	0.55	21.9443	281	11.27
2006-04-01 13	Partly Cloudy	rain	17.33333333	17.33333333	0.51	20.6885	289	11.27
2006-04-01 14	Partly Cloudy	rain	18.87777778	18.87777778	0.47	15.3755	262	11.4471
2006-04-01 15	Partly Cloudy	rain	18.91111111	18.91111111	0.46	10.4006	288	11.27
2006-04-01 16	Partly Cloudy	rain	15.38888889	15.38888889	0.6	14.4095	251	11.27
2006-04-01 17	Mostly Cloudy	rain	15.55	15.55	0.63	11.1573	230	11.4471
2006-04-01 18	Mostly Cloudy	rain	14.25555556	14.25555556	0.69	8.5169	163	11.2056
2006-04-01 19	Mostly Cloudy	rain	13.14444444	13.14444444	0.7	7.6314	139	11.2056
2006-04-01 20	Mostly Cloudy	rain	11.55	11.55	0.77	7.3899	147	11.0285
2006-04-01 21	Mostly Cloudy	rain	11.18333333	11.18333333	0.76	4.9266	160	9.982
2006-04-01 22	Partly Cloudy	rain	10.11666667	10.11666667	0.79	6.6493	163	15.8263
2006-04-01 23	Mostly Cloudy	rain	10.2	10.2	0.77	3.9284	152	14.9569
2006-04-10 00	Partly Cloudy	rain	10.42222222	10.42222222	0.62	16.9855	150	15.8263
2006-04-10 01	Partly Cloudy	rain	9.911111111	7.566666667	0.66	17.2109	149	15.8263
2006-04-10 02	Mostly Cloudy	rain	11.18333333	11.18333333	0.8	10.8192	163	14.9569
2006-04-10 03	Partly Cloudy	rain	7.155555556	5.044444444	0.79	11.0768	180	15.8263
2006-04-10 04	Partly Cloudy	rain	6.111111111	4.816666667	0.82	6.6493	161	15.8263

Case Study

❖ Temperature forecasting

Time	Temperature (C)
2006-04-01 00:00:00.000 +0200	9.472222
2006-04-01 01:00:00.000 +0200	9.355556
2006-04-01 02:00:00.000 +0200	9.377778
2006-04-01 03:00:00.000 +0200	8.288889
2006-04-01 04:00:00.000 +0200	8.755556
2006-04-01 05:00:00.000 +0200	9.222222
2006-04-01 06:00:00.000 +0200	7.733333

X

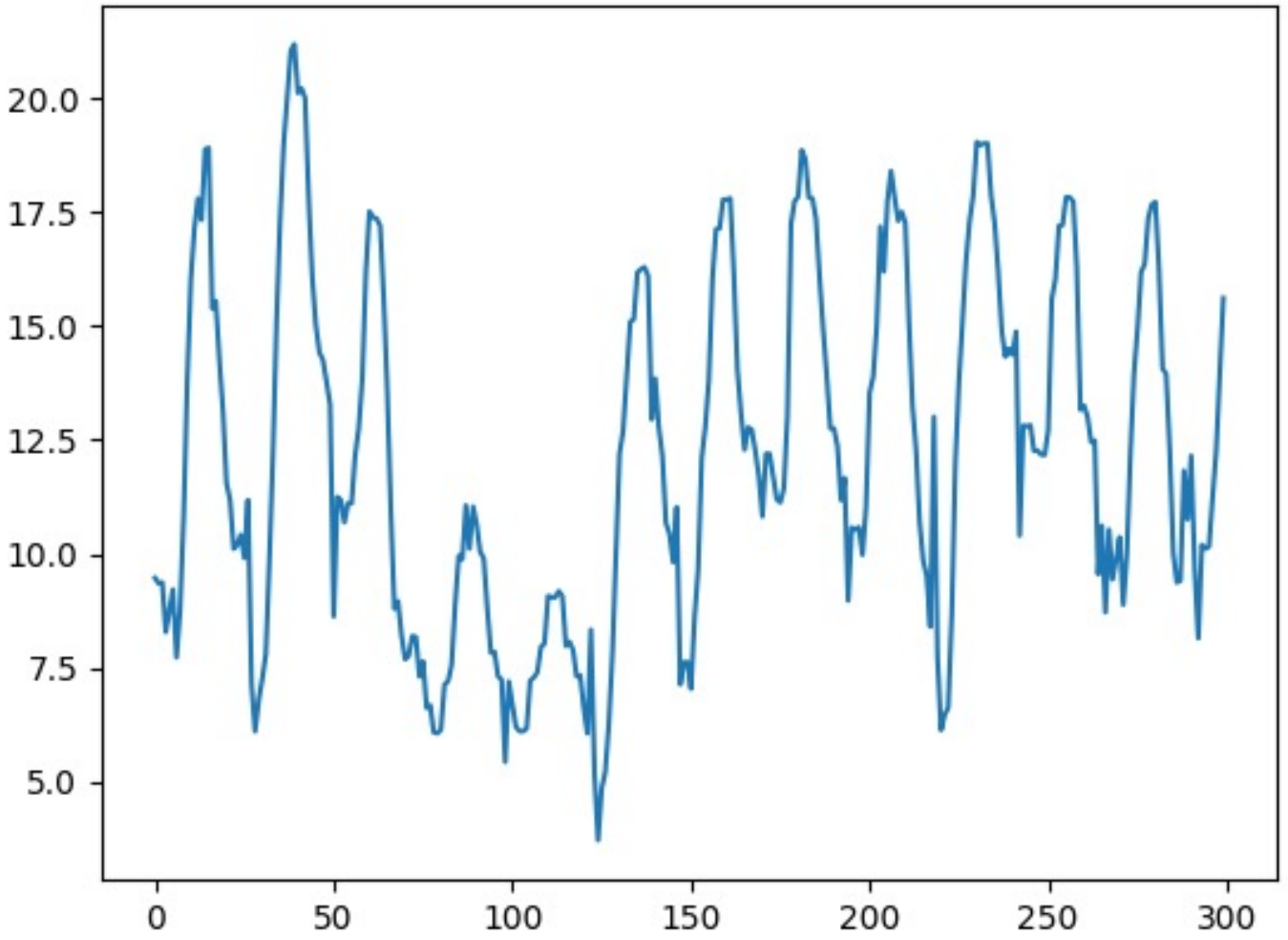
y

No Data Shuffling

Temperature forecasting datatable

Case Study

Temperature forecasting



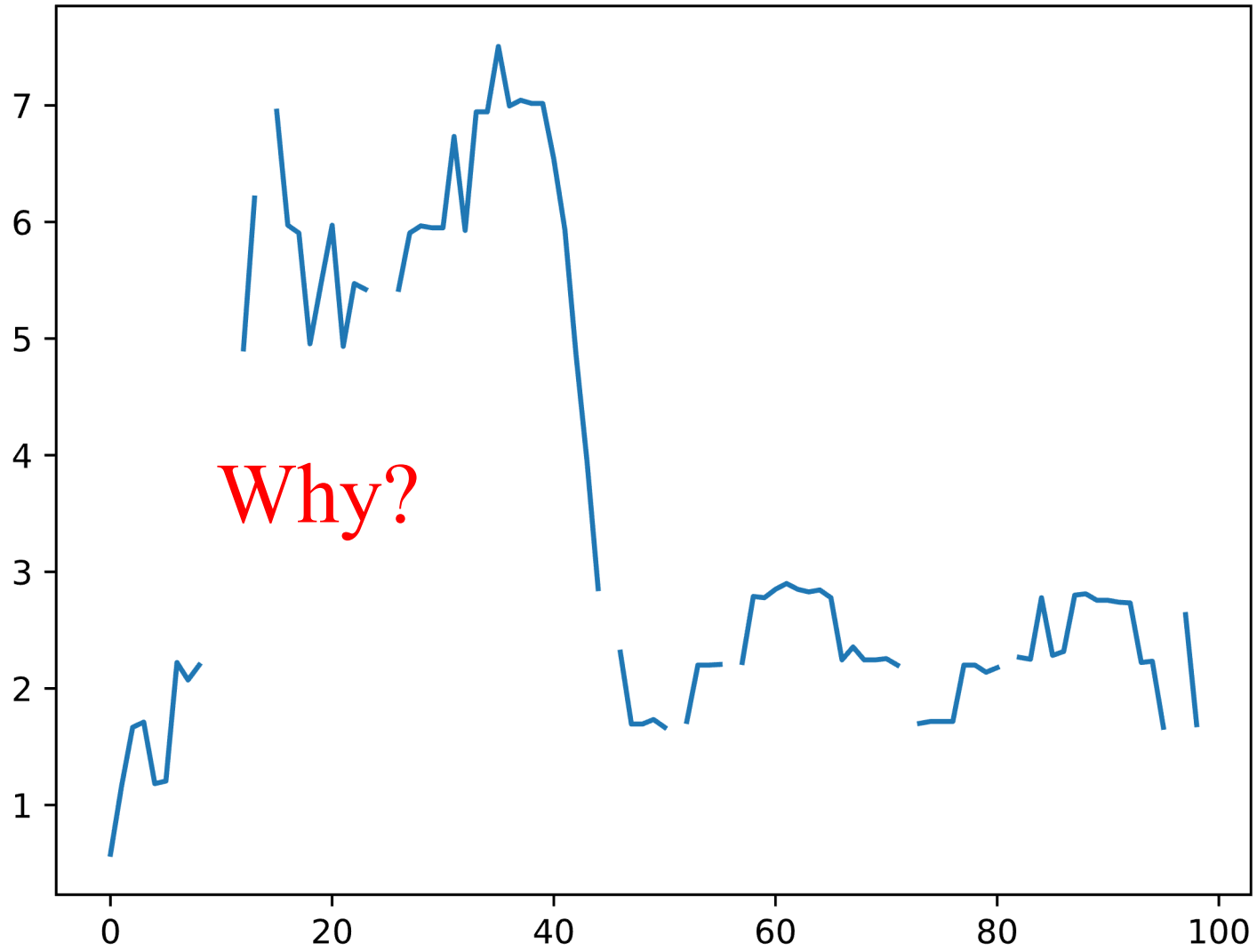
Date	Temperature (C)
2006-04-01 00	9.472222222
2006-04-01 01	9.355555556
2006-04-01 02	9.377777778
2006-04-01 03	8.288888889
2006-04-01 04	8.755555556
2006-04-01 05	9.222222222
2006-04-01 06	7.733333333
2006-04-01 07	8.772222222
2006-04-01 08	10.82222222
2006-04-01 09	13.77222222
2006-04-01 10	16.01666667
2006-04-01 11	17.14444444
2006-04-01 12	17.8
2006-04-01 13	17.33333333
2006-04-01 14	18.87777778
2006-04-01 15	18.91111111
2006-04-01 16	15.38888889
2006-04-01 17	15.55
2006-04-01 18	14.25555556
2006-04-01 19	13.14444444
2006-04-01 20	11.55
2006-04-01 21	11.18333333
2006-04-01 22	10.11666667
2006-04-01 23	10.2
2006-04-10 00	10.42222222
2006-04-10 01	9.911111111
2006-04-10 02	11.18333333
2006-04-10 03	7.155555556
2006-04-10 04	6.111111111

Temperature (C)
9.472222222
9.355555556
9.377777778
8.288888889
8.755555556
9.222222222
7.733333333
8.772222222
10.82222222
13.77222222
16.01666667
17.14444444
17.8
17.33333333
18.87777778
18.91111111
15.38888889
15.55
14.25555556
13.14444444
11.55
11.18333333
11.16666667
10.2
10.42222222
9.911111111
11.18333333
7.155555556
6.111111111

Case Study

❖ Plot the first 100 data points

Temperature (C)
0.577778
1.161111
1.666667
1.711111
1.183333
1.205556
2.222222
2.072222
2.2
NaN
2.788889
NaN
4.911111
6.205556
NaN



Case Study

❖ Missing data – Case study

Temperature (C)
0.577778
1.161111
1.666667
1.711111
1.183333
1.205556
2.222222
2.072222
2.2
NaN
2.788889
NaN
4.911111
6.205556
NaN

fillna(mean)

Temperature (C)
0.577778
1.161111
1.666667
1.711111
1.183333
1.205556
2.222222
2.072222
2.2
11.832714
2.788889
11.832714
4.911111
6.205556
11.832714

```
mean = df_train['Temperature (C)'].mean()
df_train = df_train.fillna(mean)
```

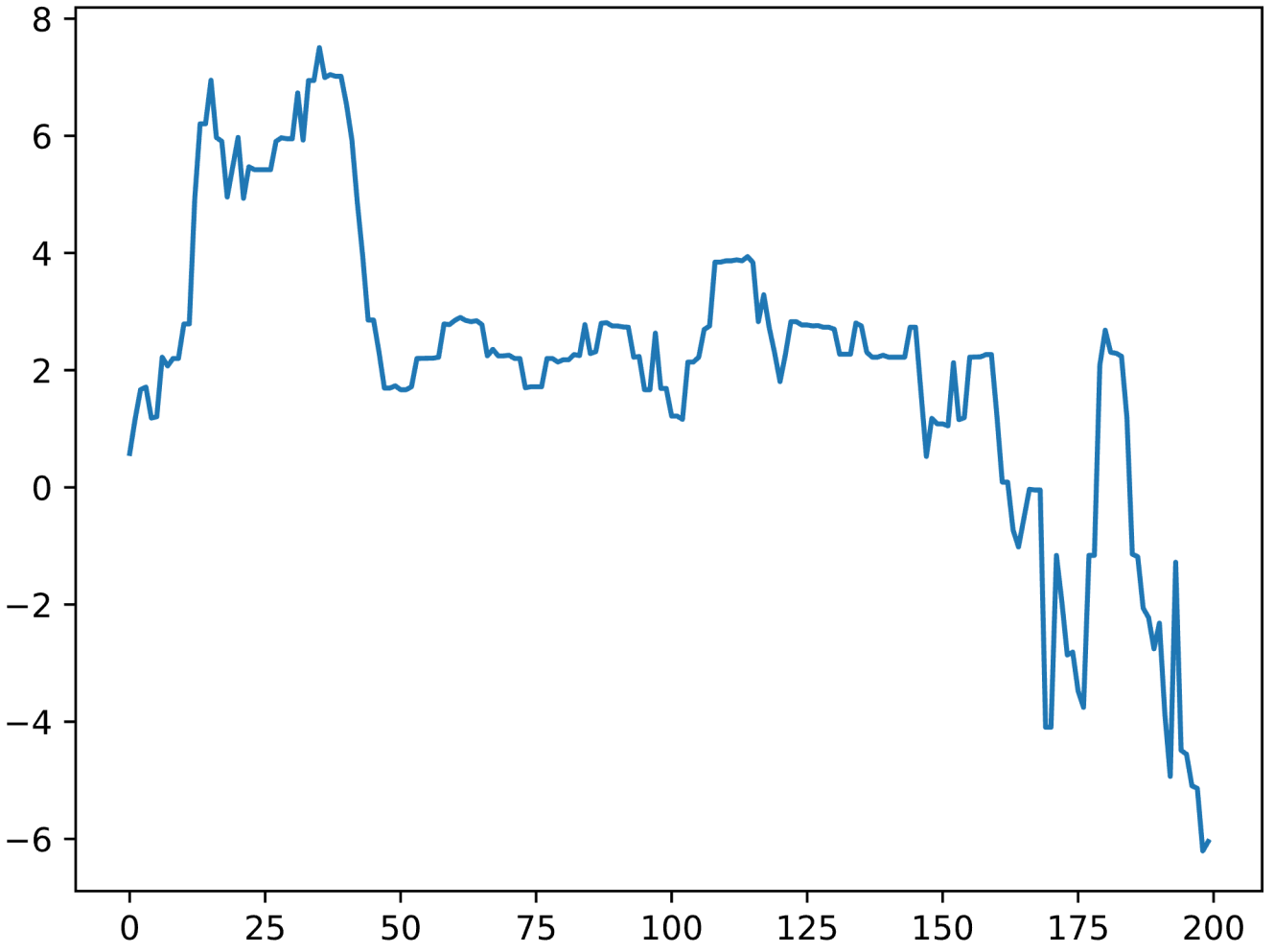
Temperature (C)
0.577778
1.161111
1.666667
1.711111
1.183333
1.205556
2.222222
2.072222
2.2
NaN
2.788889
NaN
4.911111
6.205556
NaN

interpolate()

Temperature (C)
0.577778
1.161111
1.666667
1.711111
1.183333
1.205556
2.222222
2.072222
2.2
2.494444
2.788889
3.85
4.911111
6.205556
6.577778

```
df_train = df_train.interpolate()
```

Noise Reduction



How to Use historical Data

❖ Moving average

$k = 2$

3	8	6	5	1	7	9	0	8	4
5.5	7.0	5.5	3.0	4.0	8.0	4.5	4.0	6.0	

Simple Moving Average (SMA)

$$SMA_t = \frac{s_t + \cdots + s_{t-k+1}}{k}$$

3	8	6	5	1	7	9	0	8	4
3.0	5.5	5.8	5.4	3.2	5.1	7.0	3.5	5.8	4.9

Exponential moving average (EMA)

(Exponential weighted average)

$$EMA_t = \rho EMA_{t-1} + (1 - \rho)s_t$$

How to Use historical Data

❖ Exponentially weighted averages

3	8	6	5	1	7	9	0	8	4
3.0	5.5	5.8	5.4	3.2	5.1	7.0	3.5	5.8	4.9

$$V_t = \rho V_{t-1} + (1-\rho)s_t$$

$$V_1 = \rho V_0 + (1-\rho)s_1$$

$$V_2 = \rho V_1 + (1-\rho)s_2$$

$$V_3 = \rho V_2 + (1-\rho)s_3$$

$$V_3 = \rho[\rho V_1 + (1-\rho)s_2] + (1-\rho)s_3$$

$$V_3 = \rho[\rho[\rho V_0 + (1-\rho)s_1] + (1-\rho)s_2] + (1-\rho)s_3$$

Given $V_0 = 0$, we have

$$V_3 = \rho[\rho(1-\rho)s_1 + (1-\rho)s_2] + (1-\rho)s_3$$

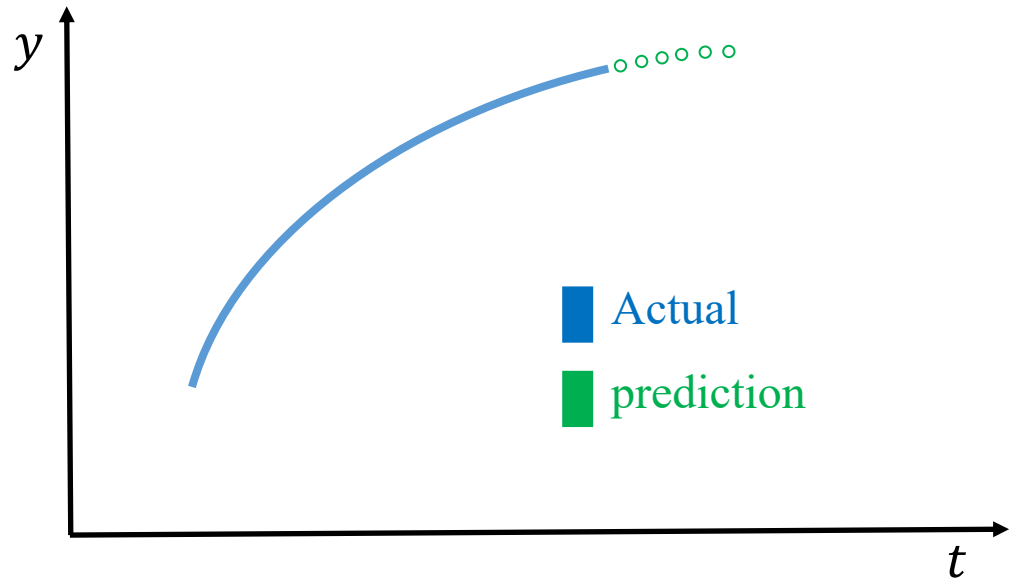
$$V_3 = \rho^2(1-\rho)s_1 + \rho(1-\rho)s_2 + (1-\rho)s_3$$

How to Use historical Data

❖ Exponentially weighted averages

$$V_t = \rho V_{t-1} + (1-\rho)s_t$$

data	3	8	6	5	1	7	9	0	8	4
EWA	3.0	5.5	5.8	5.4	3.2	5.1	7.0	3.5	5.8	4.9



Example

$$V_3 = \rho^2 (1-\rho)s_1 + \rho(1-\rho)s_2 + (1-\rho)s_3$$

With $\rho = 0.5$

$$V_3 = 0.125s_1 + 0.25s_2 + 0.5s_3$$

With $\rho = 0.9$

$$V_3 = 0.081s_1 + 0.09s_2 + 0.1s_3$$

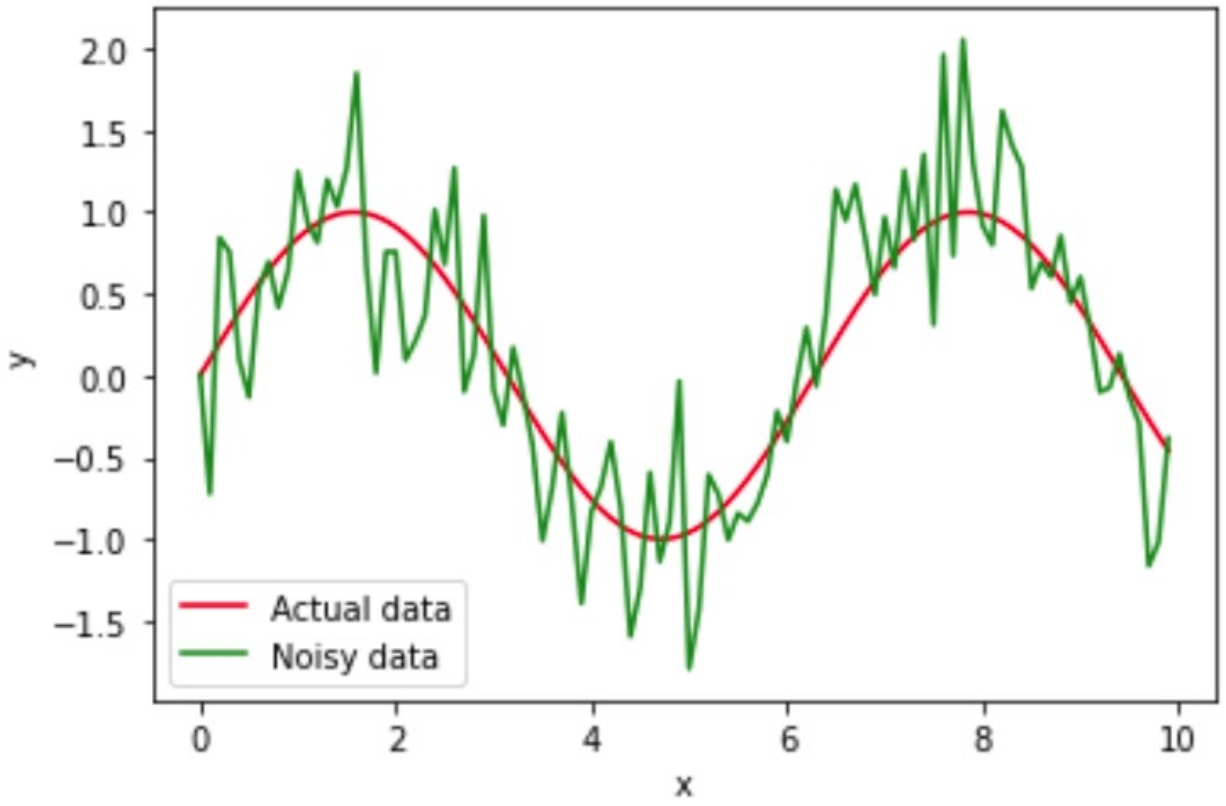
With $\rho = 0.98$

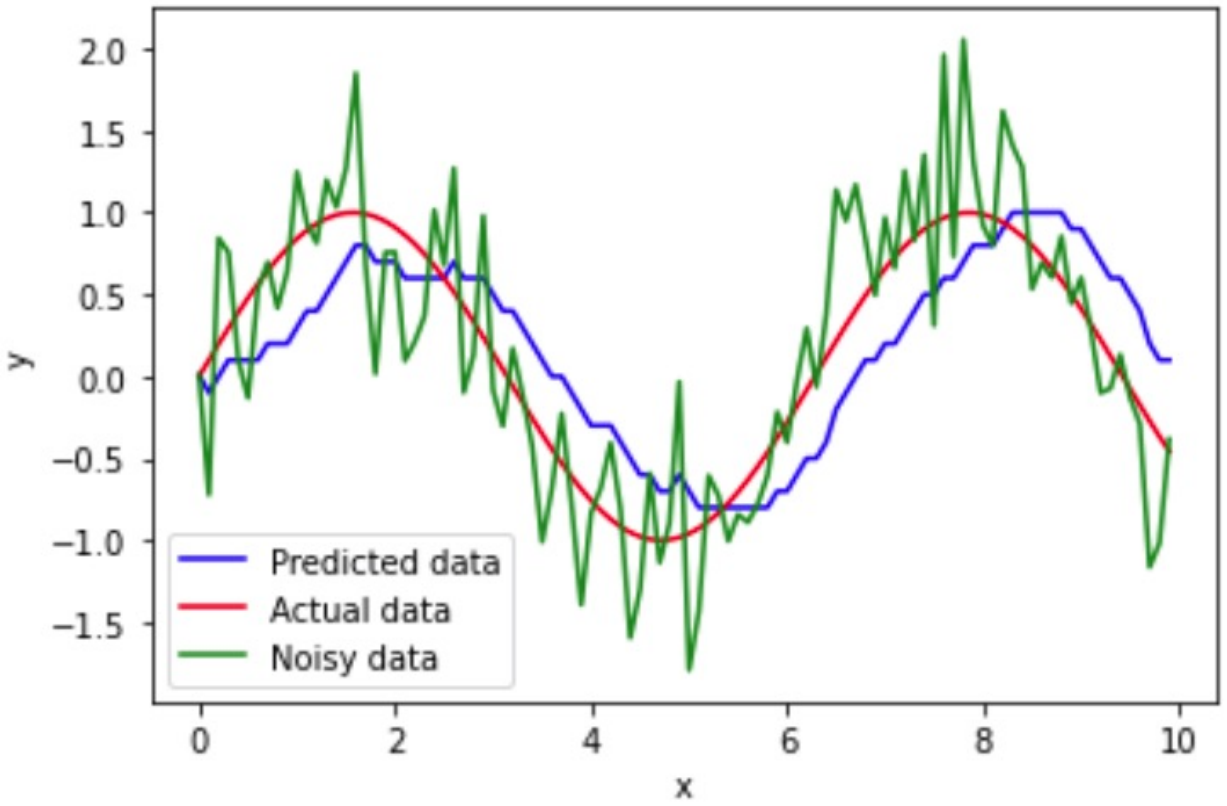
$$V_3 = 0.0392s_1 + 0.0196s_2 + 0.02s_3$$

Noise Reduction

❖ Exponentially weighted averages

$\rho = 0.9$

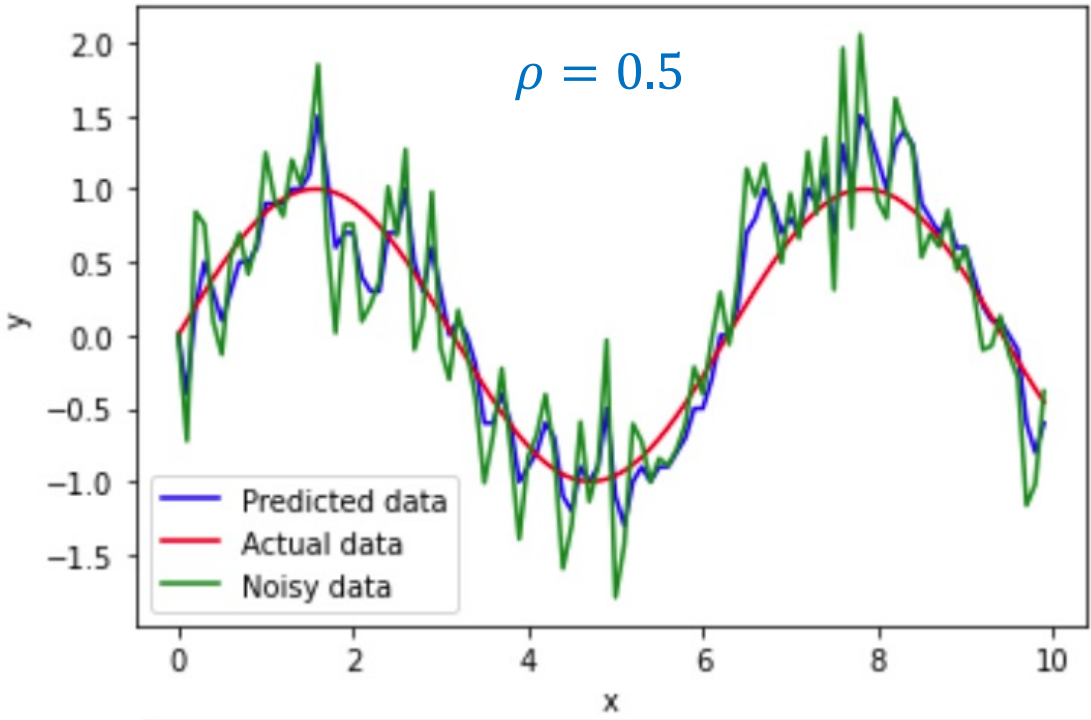


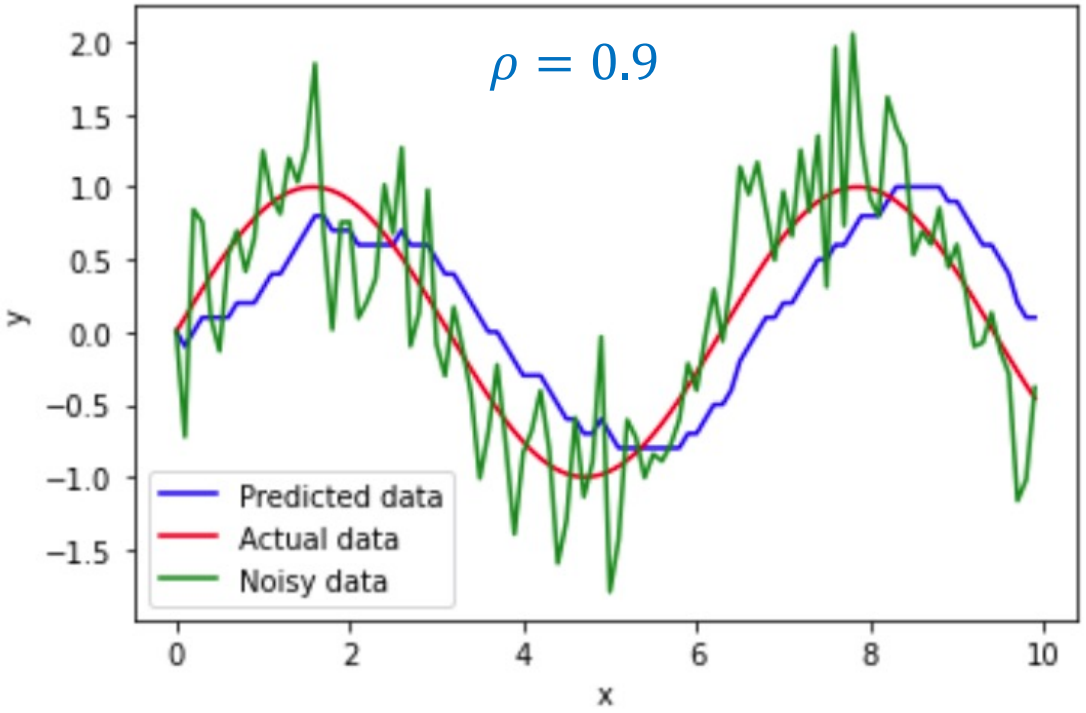


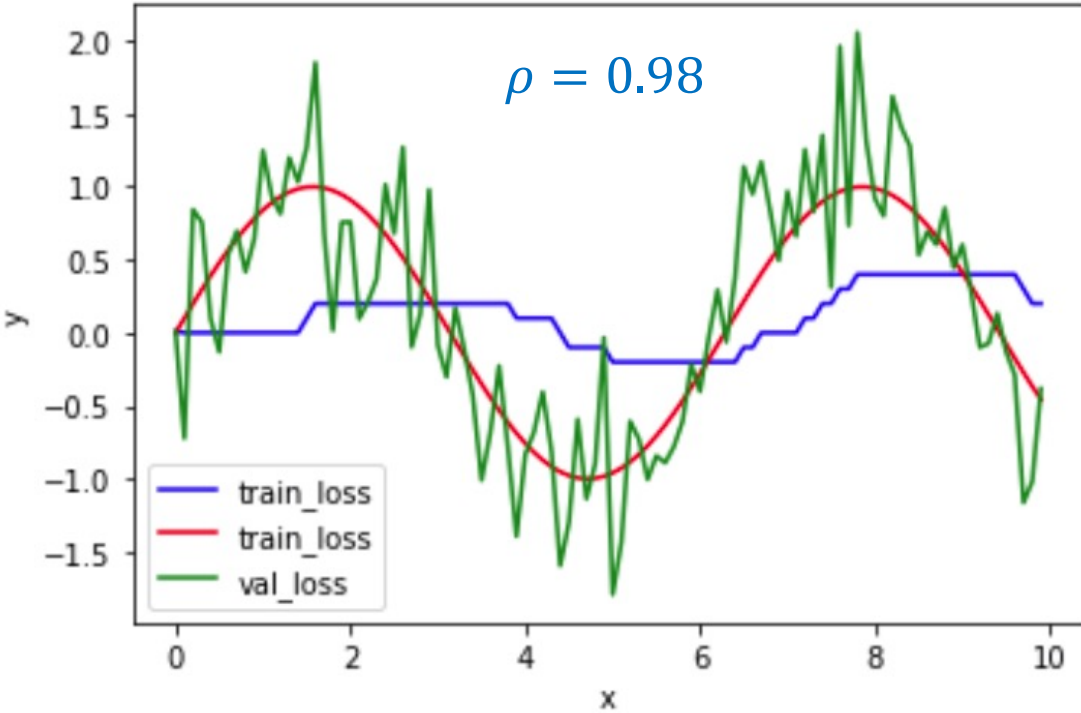
Noise Reduction

❖ **Exponentially weighted averages**

$$V_t = \rho V_{t-1} + (1-\rho)s_t$$



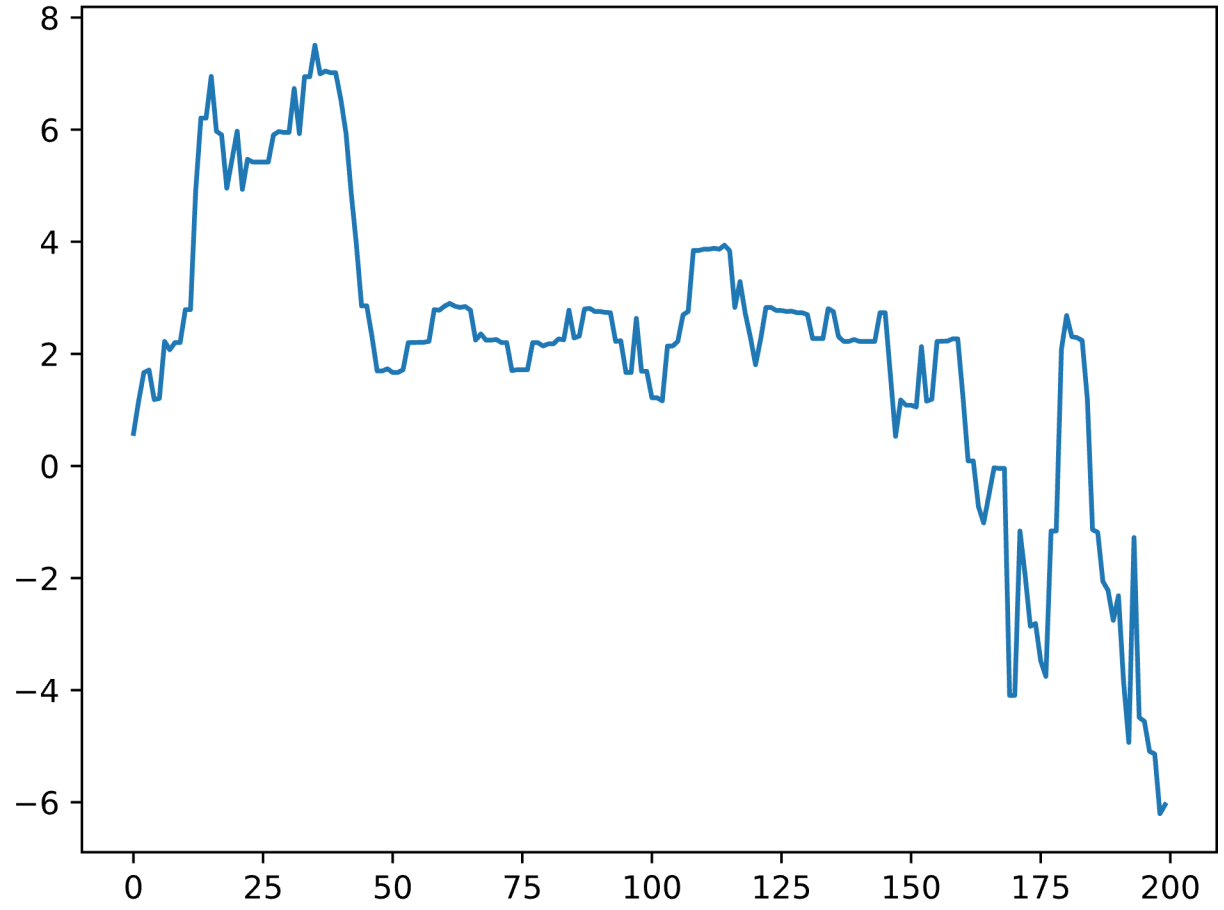


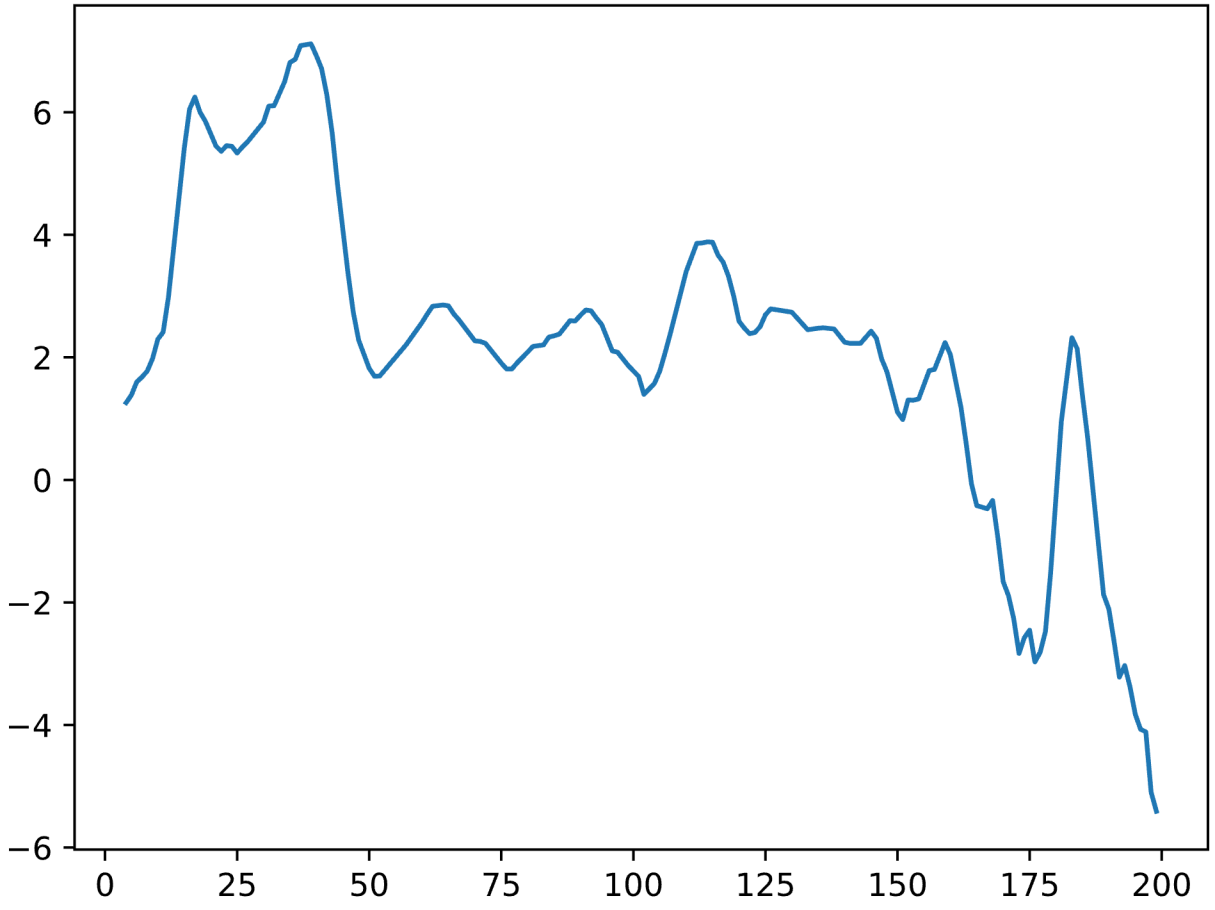


Noise Reduction

❖ Apply to the weather data

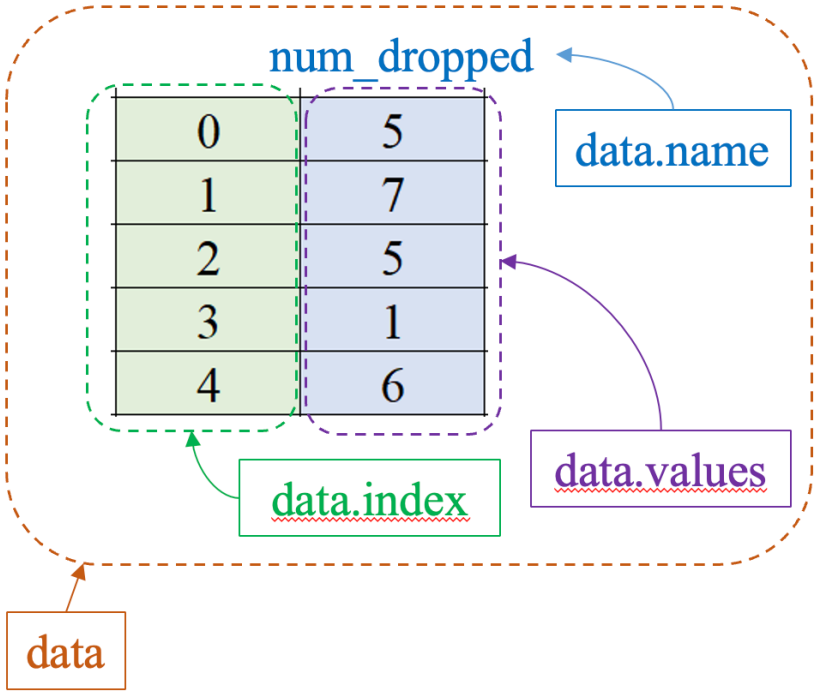
data.rolling(window=5).mean()



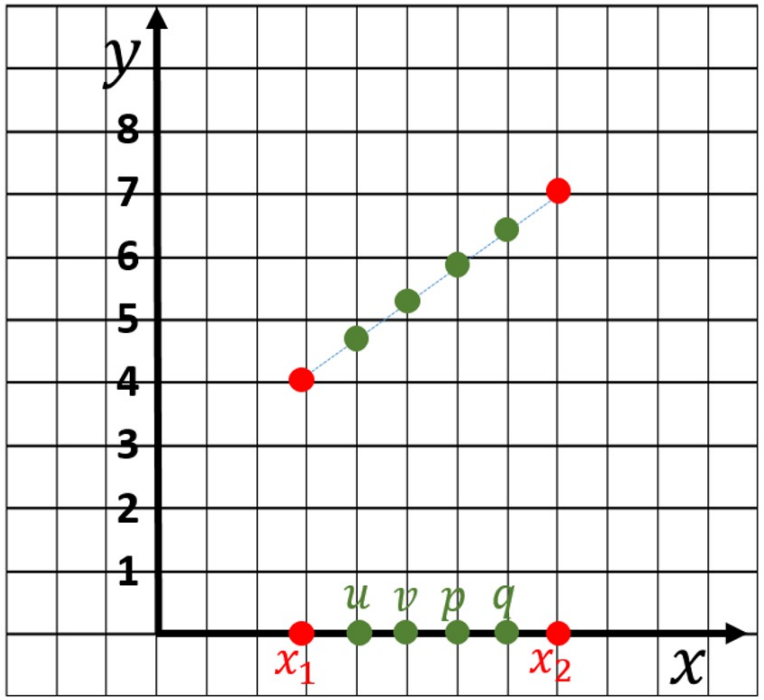


Summary

Series in Pandas



Operations



Nội suy theo hàm tuyến tính

Case Studies